

Claude Code Source Leak - Complete Analysis & Key Code

On March 31, 2026, someone at Anthropic forgot to add *.map to .npmignore. That one missing line exposed 512,000+ lines of TypeScript source code — the entire Claude Code codebase. This document collects the most important files, analysis, and explanations from the leak.

Table of Contents

- 1. **Project Overview** - README summary with architecture, tool system, and comparison table
- 2. **Architecture Analysis** - How nano-claude-code (Python rewrite) works internally
- 3. **Claude Code vs ClawSpring Comparison** - Side-by-side strengths and weaknesses
- 4. **Undercover Mode** - How Anthropic employees hide AI contributions to open source
- 5. **System Prompt Sections** - What instructions Anthropic injects into every conversation
- 6. **Tool Constants** - The complete list of 40+ tools Claude Code exposes
- 7. **Penguin Mode (Fast Mode)** - Internal fast mode implementation with kill-switches
- 8. **Auto-Dream** - Memory consolidation system that runs during idle time
- 9. **Context Compaction** - The 5 compaction strategies for managing context limits

1. Project Overview

This repository (`chauncygu/collection-claude-code-source-code`) contains subprojects studying Claude Code from multiple angles:

Subproject	Language	Nature	File Count
original-source-code	TypeScript	Raw leaked source archive	1,884 files

claude-code-source-code	TypeScript	Decompiled source (v2.1.88) + docs	1,940 files
claw-code	Python	Clean-room architectural rewrite	109 files
nano-claude-code	Python	Minimal multi-provider reimplementation	~30 files

Scale of the leak: ~1,900 TypeScript files, 512,000+ lines of code, 40+ registered tools, 44 hidden feature flags, 5 context compaction strategies, 25+ event hooks, 3 subagent execution models (Fork, Teammate, Worktree).

Key discoveries: Undercover Mode (stealth for Anthropic employees), KAIROS (always-on autonomous agent), autoDream (idle-time memory consolidation), anti-distillation (poisoning competitor training data), model codenames (Capybara, Fennec, Numbat, Opus 4.7, Sonnet 4.8), and 6+ emergency kill-switches.

The full README is available at: <https://github.com/chauncygu/collection-claude-code-source-code/blob/main/README.MD>

2. Architecture Analysis

This document explains the complete architecture of nano-claude-code, a Python rewrite of Claude Code. It reveals how the agent loop works, the tool registry pattern, context compaction, memory system, sub-agent lifecycle, and providers abstraction.

Architecture Guide

This document is for developers who want to understand, modify, or extend clawspring.

For user-facing docs, see [README.md](../README.md).

Overview

Nano-claude-code is a ~3.4K-line Python CLI that lets LLMs (GPT, Gemini, etc.) operate as coding agents with tool use, memory, sub-agents, and skills. The architecture is a flat module layout designed for readability and future migration to a package structure.

User Input

|



clawspring.py — REPL, slash commands, rendering

|

| —> agent.py — multi-turn loop, permission gates

| |

| | —> providers.py — API streaming (Anthropic / OpenAI-compatible)

| | —> tool_registry.py —> tools.py — 13 tools

| | —> compaction.py — context window management

| | —> subagent.py — threaded sub-agent lifecycle

|

| —> context.py — system prompt (git, CLAUDE.md, memory)

| | —> memory.py — persistent file-based memory

|

| —> skills.py — markdown skill loading + execution

| —> config.py — configuration persistence

```

**Key invariant:** Dependencies flow downward. No circular imports at
the module level
(subagent.py uses lazy imports to call agent.py).

- - -

## Module Reference

### `tool_registry.py` – Tool Plugin System

The central registry that all tools register into. This is the
foundation for extensibility.

**Data model:**

```python
@dataclass
class ToolDef:
 name: str # unique identifier (e.g. "Read",
 "MemorySave")
 schema: dict # JSON schema sent to the LLM API
 func: Callable # (params: dict, config: dict) -> str
 read_only: bool # True = auto-approve in 'auto' permission
mode
 concurrent_safe: bool # True = safe to run in parallel (for sub-
agents)

```

## Public API:

Function	Description
<code>register_tool(tool_def)</code>	Add a tool to the registry (overwrites by name)
<code>get_tool(name)</code>	Look up by name, returns <code>None</code> if not found

<code>get_all_tools()</code>	List all registered tools
<code>get_tool_schemas()</code>	Return schemas for API calls
<code>execute_tool(name, params, config, max_output=32000)</code>	Execute with output truncation
<code>clear_registry()</code>	Reset — for testing only

**Output truncation:** If a tool returns more than `max_output` chars, the result is truncated to `first_half + [... N chars truncated ...] + last_quarter`. This prevents a single tool call (e.g. reading a huge file) from blowing up the context window.

### Registering a custom tool:

```
from tool_registry import ToolDef, register_tool

def my_tool(params, config):
 return f"Hello, {params['name']}!"

register_tool(ToolDef(
 name="MyTool",
 schema={
 "name": "MyTool",
 "description": "A greeting tool",
 "input_schema": {
 "type": "object",
 "properties": {"name": {"type": "string"}},
 "required": ["name"],
 },
 },
 func=my_tool,
 read_only=True,
 concurrent_safe=True,
))
```

## `tools.py` — Built-in Tool Implementations

Contains the 8 core tools (Read, Write, Edit, Bash, Glob, Grep, WebFetch, WebSearch) plus memory tools (MemorySave, MemoryDelete) and sub-agent tools (Agent, CheckAgentResult, ListAgentTasks). All register themselves via `tool_registry` at import time.

### Key internals:

- `_is_safe_bash(cmd)` — whitelist of safe shell commands for auto-approval
- `generate_unified_diff(old, new, filename)` — diff generation for Edit/Write
- `maybe_truncate_diff(diff_text, max_lines=80)` — truncate large diffs for display
- `_get_agent_manager()` — lazy singleton for SubAgentManager
- Backward-compatible `execute_tool(name, inputs, permission_mode, ask_permission)` wrapper

## `agent.py` — Core Agent Loop

The heart of the system. `run()` is a generator that yields events as they happen.

```
def run(user_message, state, config, system_prompt,
 depth=0, cancel_check=None) -> Generator:
```

### Loop logic:

1. Append user message
2. Inject depth into config (for sub-agent depth tracking)
3. While True:
  - a. Check `cancel_check()` — cooperative cancellation for sub-agents
  - b. `maybe_compact(state, config)` — compress if near context limit
  - c. Stream from provider → yield `TextChunk` / `ThinkingChunk`
  - d. Record assistant message
  - e. If no `tool_calls` → break
  - f. For each `tool_call`:
    - Permission check (`_check_permission`)
    - If denied → yield `PermissionRequest` → user decides
    - Execute tool → yield `ToolStart` / `ToolEnd`
    - Append tool result
  - g. Loop (model sees tool results and responds)

## Event types:

Event	Fields	When
TextChunk	text	Streaming text delta
ThinkingChunk	text	Extended thinking block
ToolStart	name, inputs	Before tool execution
ToolEnd	name, result, permitted	After tool execution
PermissionRequest	description, granted	Needs user approval
TurnDone	input_tokens, output_tokens	End of one API turn

## compaction.py — Context Window Management

Keeps conversations within model context limits using two layers.

### Layer 1: Snip ( snip\_old\_tool\_results )

- Rule-based, no API cost
- Truncates tool-role messages older than `preserve_last_n_turns` (default 6)
- Keeps first half + last quarter of the content

### Layer 2: Auto-Compact ( compact\_messages )

- Model-driven: calls the current model to summarize old messages
- Splits messages into [old | recent] at ~70/30 ratio
- Replaces old messages with a summary + acknowledgment

**Trigger:** `maybe_compact()` checks `estimate_tokens(messages) > context_limit * 0.7`.

Runs snip first (cheap), then auto-compact if still over.

**Token estimation:** `len(content) / 3.5` — simple heuristic. Works for most models.  
`get_context_limit(model)` reads from the provider registry.

## memory.py — Persistent Memory

File-based memory system stored in `~/.clawspring/memory/`.

## Storage format:

```
~/.clawspring/memory/
├─ MEMORY.md # Index: one line per memory
├─ user_preferences.md # Individual memory file
└─ project_auth.md
```

Each memory file uses markdown with YAML frontmatter:

```

name: user preferences
description: coding style preferences
type: feedback
created: 2026-04-02

User prefers 4-space indentation and type hints.
```

## How it integrates:

- `get_memory_context()` returns the MEMORY.md index text
- `context.py` injects this into the system prompt
- The model reads the index, then uses `Read` tool to access full memory content
- The model uses `MemorySave` / `MemoryDelete` tools to manage memories

## subagent.py — Threaded Sub-Agents

Sub-agents run in background threads via `ThreadPoolExecutor`.

### Key design decisions:

1. **Fresh context** — each sub-agent starts with empty message history + task prompt
2. **Depth limiting** — `max_depth=3`, checked at spawn time. Model gets an error message (not silent tool removal) so it can adapt.
3. **Cooperative cancellation** — `cancel_check` callable checked each loop iteration. Python threads can't be killed safely, so we set a flag.
4. **Threading, not asyncio** — the entire codebase is synchronous generators. Threading via `concurrent.futures` keeps things simple. The SubAgentManager API is designed to

be compatible with a future async migration.

### Lifecycle:

```
spawn(prompt, config, system_prompt, depth)
 → Creates SubAgentTask
 → Submits _run to ThreadPoolExecutor
 → _run calls agent.run() with depth+1

wait(task_id, timeout) → blocks until complete
cancel(task_id) → sets _cancel_flag
get_result(task_id) → returns result string
```

## skills.py — Reusable Prompt Templates

Skills are markdown files with frontmatter. They are **not code** — just structured prompts that get injected into the agent loop.

### Skill file format:

```

name: commit
description: Create a conventional commit
triggers: ["/commit"]
tools: [Bash, Read]

Your prompt instructions here...
```

**Execution:** `execute_skill()` wraps the skill prompt as a user message and calls `agent.run()`. The skill runs through the exact same agent loop as a normal query.

**Search order:** Project-level (`./.clawspring/skills/`) overrides user-level (`~/.clawspring/skills/`) when skill names collide.

## providers.py — Multi-Provider Abstraction

Two streaming adapters cover all providers:

Adapter	Providers
<code>stream_anthropic()</code>	Anthropic (native SDK)
<code>stream_openai_compat()</code>	OpenAI, Gemini, Kimi, Qwen, Zhipu, DeepSeek, Ollama, LM Studio, Custom

**Neutral message format** (provider-independent):

```
{"role": "user", "content": "..."}
{"role": "assistant", "content": "...", "tool_calls": [{"id": "...",
"name": "...", "input": {...}]}
{"role": "tool", "tool_call_id": "...", "name": "...", "content": "..."}

```

Conversion functions: `messages_to_anthropic()` , `messages_to_openai()` , `tools_to_openai()` .

**Provider-specific handling:**

- Gemini 3 models require `thought_signature` in tool call responses — this is transparently captured and passed through via `extra_content` on `tool_call` dicts.

**context.py — System Prompt Builder**

Assembles the system prompt from:

1. Base template (role, date, cwd, platform)
2. Git info (branch, status, recent commits)
3. CLAUDE.md content (project-level + global)
4. Memory index (from `memory.get_memory_context()` )

**config.py — Configuration**

Defaults stored in `~/.clawspring/config.json` . Key settings:

Key	Default	Description
<code>model</code>	<code>claude-opus-4-6</code>	Active model

max_tokens	8192	Max output tokens
permission_mode	auto	Permission mode
max_tool_output	32000	Tool output truncation limit
max_agent_depth	3	Max sub-agent nesting
max_concurrent_agents	3	Thread pool size

## Data Flow Example

A user asks "Read config.py and change max\_tokens to 16384":

```
1. clawspring.py captures input
2. agent.run() appends user message, calls maybe_compact()
3. providers.stream() sends to Gemini API with 13 tool schemas
4. Model responds: text + tool_call[Read(config.py)]
5. agent.py checks permission (Read = read_only → auto-approve)
6. tool_registry.execute_tool("Read", ...) → file content (truncated if >32K)
7. Tool result appended to messages, loop back to step 3
8. Model responds: text + tool_call[Edit(config.py, "8192", "16384")]
9. agent.py checks permission (Edit = not read_only → ask user)
10. User approves → tools.py._edit() runs, generates diff
11. clawspring.py renders diff with ANSI colors (red/green)
12. Tool result appended, loop back to step 3
13. Model responds: "Done, max_tokens changed to 16384"
14. No tool_calls → loop ends, TurnDone yielded
```

## Testing

```
Run all 78 tests
python -m pytest tests/ -v

Run specific module tests
python -m pytest tests/test_tool_registry.py -v
python -m pytest tests/test_compaction.py -v
python -m pytest tests/test_memory.py -v
python -m pytest tests/test_subagent.py -v
python -m pytest tests/test_skills.py -v
python -m pytest tests/test_diff_view.py -v
```

Tests use `monkeypatch` and `tmp_path` fixtures to avoid side effects.  
Sub-agent tests mock `_agent_run` to avoid real API calls.

---

## Future: Package Refactoring

When `tools.py` or `agent.py` grow too large, the flat layout can be migrated to:

```
ncc/
├─ __init__.py
├─ repl.py # from clawspring.py
├─ agent/
│ ├─ loop.py # from agent.py
│ ├─ subagent.py # from subagent.py
│ └─ compaction.py # from compaction.py
├─ providers/
│ ├─ base.py
│ ├─ openai_compat.py
│ └─ registry.py
├─ tools/
│ ├─ registry.py # from tool_registry.py
│ ├─ builtin.py # core 8 tools from tools.py
│ ├─ memory.py # MemorySave/MemoryDelete from tools.py
│ └─ subagent.py # Agent/Check/List from tools.py
├─ memory/
│ └─ store.py # from memory.py
├─ skills/
│ └─ loader.py # from skills.py
└─ config.py
```

The current code is structured to make this migration straightforward:

- Modules communicate via function parameters, not globals
- Each module has a small public API surface
- Dependencies are unidirectional

---

## ## 3. Claude Code vs ClawSpring Comparison

> Side-by-side comparison of the original TypeScript Claude Code (283K lines, 1,332 files, 44+ tools, Anthropic-only) vs the Python rewrite (10K lines, 51 files, 21 tools, multi-provider). Key finding: Claude Code is engineering-complete but single-provider. ClawSpring is minimal but extensible.

```markdown

Claude Code vs ClawSpring v3.03: Comparative Analysis

Overview

| Dimension | Claude Code (TypeScript) | ClawSpring (Python) |
|---------------------|--------------------------|---------------------|
| **Language** | TypeScript + React/Ink | Python 3.8+ |
| **File Count** | 1,332 TS/TSX files | 51 Python files |
| **Code Size** | ~283K lines | ~10.2K lines |
| **Tool Count** | 44+ tools | 21 tools |
| **Slash Commands** | 88 | 17 |
| **Model Providers** | Anthropic only | 7+ providers |

Architecture Comparison

Agent Loop (Core)

Claude Code – `QueryEngine.ts` (~47KB)

- Complex message stream management with extended thinking support
- Multi-layer permission system (MDM, team management, per-tool gates)
- Built-in context compaction scheduler

ClawSpring – `agent.py` (174 lines)

- Generator-based event stream (`yield TextChunk | ToolStart | ToolEnd`)
- Clean event flow, highly readable code
- Cooperative cancellation support for sub-agents

Tool System

****Claude Code****

- Each tool in its own directory with full schema, permission docs, and tests
- Complex inter-tool dependencies (EnterWorktree → isolated env → ExitWorktree)
- Advanced tools: `SyntheticOutputTool`, `RemoteTriggerTool`, etc.

****ClawSpring****

- `ToolDef` registry pattern – any module can dynamically inject via `register\_tool()`
- `read\_only` / `concurrent\_safe` flags drive automatic permission decisions
- Output truncation (32KB cap) prevents context bloat

UI Rendering

****Claude Code****: React + Ink, full component tree, diff visualization, dialogs, progress bars, streaming rendering

****ClawSpring****: `rich` library, Markdown syntax highlighting – simple but sufficient

Strengths and Weaknesses

Claude Code Strengths

1. ****Engineering completeness**** – 88 slash commands, 44 tools, enterprise-grade permission management (MDM/team config)
2. ****UI quality**** – React/Ink component tree, high-quality streaming rendering, fine-grained diff visualization

3. ****Service layer depth**** – LSP integration, Analytics/Telemetry, OAuth, GrowthBook feature flags
4. ****Session memory**** – AI-driven memory extraction (`extractMemories` service)
5. ****Production reliability**** – Single bundled `cli.js` (12MB), comprehensive test coverage, version-locked

Claude Code Weaknesses

1. ****Single provider**** – Anthropic API only, no OpenAI/Gemini/local model support
2. ****Poor code readability**** – 1,332 files + decompiled code, scattered logic, hard to modify
3. ****Heavy build dependency**** – Bun + esbuild + TypeScript; changes require a full build chain
4. ****Closed extensibility**** – Feature flags (`internal-only modules`) dead-code-eliminated at compile time, no external extension

ClawSpring Strengths

1. ****Multi-provider**** – Anthropic/OpenAI/Gemini/Kimi/Qwen/DeepSeek/Ollama with auto-detection
2. ****Highly readable**** – 51 files, 10K lines, architecture immediately apparent, great for learning and research
3. ****Zero build**** – Pure Python, `pip install` ready, changes take effect immediately
4. ****Dynamic extensibility**** – Runtime injection via `register\_tool()`, Plugin system supports git URL installation
5. ****Markdown Skills**** – Custom skills via `~/clawspring/skills/\*.md` without touching source code
6. ****Task dependency graph**** – `task/store.py` has `blocks/blocked\_by` dependency tracking (absent in Claude Code)

ClawSpring Weaknesses

1. ****Thin UI**** – No diff visualization, no dialog system, no progress bar components
2. ****Tool coverage gaps**** – Missing `WebSearch` (needs API key), `NotebookEdit`, `LSP Diagnostics`
3. ****Weak security posture**** – No MDM control, no team permission sync,

no keychain integration

4. **Performance** – Slower Python startup, large file handling slower than Node.js

5. **Immature sub-agents** – `subagent.py` still has stubs; multi-agent coordination less stable than the original

Key Design Differences

Context Compaction Strategy

| | Claude Code | ClawSpring |
|--------------------|---|-------------------------------------|
| Trigger | Exact token counting | `len/3.5` estimate at 70% threshold |
| Compression layers | Single-layer AI summarization (rule-based) + AI summary | Two layers: Snip + AI summary |
| Retention policy | System-level scheduler | `preserve_last_n_turns` parameter |

Memory System

Claude Code's `extractMemories` service uses AI to proactively extract knowledge from conversations. Nano's `memory/` module is tool-driven explicit storage (`MemorySave` tool calls) – more controllable but requires the model to actively use it.

Module Structure

Claude Code (services layer):

```
services/  
├── api/ # Claude API, OAuth, analytics  
├── mcp/ # Model Context Protocol client  
├── plugins/ # Plugin installation/execution  
├── SessionMemory/ # Persistent session state  
└── compact/ # Context compression
```

- |—— lsp/ # Language Server Protocol
- |—— extractMemories/ # AI-driven memory extraction
- |—— MagicDocs/ # Documentation generation

****ClawSpring**** (flat packages):

- mcp/ # MCP client (stdio/SSE/HTTP)
- memory/ # Persistent dual-scope memory
- task/ # Task management with dependency graph
- skill/ # Markdown-based skill loader
- multi_agent/ # Sub-agent lifecycle + git worktrees
- plugin/ # Plugin loader/recommend system

Supported Models

Claude Code: Anthropic only (`claude-opus-4-6`, `claude-sonnet-4-6`,
`claude-haiku-4-5`)

ClawSpring:

| Provider | Models |
|----------------|--|
| Anthropic | claude-opus-4-6, claude-sonnet-4-6, claude-haiku-4-5 |
| OpenAI | gpt-4o, gpt-4o-mini, o1, o3-mini |
| Google | gemini-2.5-pro, gemini-2.0-flash, gemini-1.5-pro |
| Moonshot | kimi-latest |
| Alibaba | qwen-max, qwen-plus |
| Zhipu | glm-4 |
| DeepSeek | deepseek-chat, deepseek-reasoner |
| Ollama (local) | llama3.3, qwen2.5-coder, deepseek-r1, mistral, phi4, codellama |

Summary

ClawSpring is a **minimal reimplementation** of Claude Code's core philosophy. It surpasses the original in multi-provider support and code readability, making it ideal as a research foundation or personal tool. Claude Code is a complete engineering product with unmatched UI, security, and enterprise features. The most valuable gaps to bridge are: **WebSearch tool**, **more robust sub-agent coordination**, and **diff visualization rendering**.

| Gap | Priority | Notes |
|----------------|----------|----------------------------|
| WebSearch tool | High | Needs a search API backend |

```
Diff visualization	Medium	Rich has diff support
Sub-agent stability	Medium	`subagent.py` stubs need fleshing out
LSP diagnostics	Low	Requires language server per language
Keychain integration	Low	`keyring` library available for Python
```

4. Undercover

UNDERCOVER MODE: The most controversial discovery from the leak. When Anthropic employees (`USER_TYPE == 'ant'`) use Claude Code on public/open-source repositories, this mode automatically activates. It strips all "Co-Authored-By" attribution from commits (removing any trace that Claude Code generated the code), forbids mentioning internal codenames like Opus 4.7 or Sonnet 4.8, blocks references to Slack channels or internal repo names, and even forbids the phrase "Claude Code" itself. There is NO force-OFF mechanism for external users. In external builds shipped to regular users, the entire undercover function is dead-code-eliminated at compile time. The ethical concern: AI-generated code enters open-source repositories without attribution to either the AI or the company.

```

/**
 * Undercover mode – safety utilities for contributing to public/open-
source repos.
 *
 * When active, Claude Code adds safety instructions to commit/PR
prompts and
 * strips all attribution to avoid leaking internal model codenames,
project
 * names, or other Anthropic-internal information. The model is not told
what
 * model it is.
 *
 * Activation:
 *   - CLAUDE_CODE_UNDERCOVER=1 – force ON (even in internal repos)
 *   - Otherwise AUTO: active UNLESS the repo remote matches the
internal
 *     allowlist (INTERNAL_MODEL_REPOS in commitAttribution.ts). Safe
default
 *     is ON – Claude may push to public remotes from a CWD that isn't
itself
 *     a git checkout (e.g. /tmp crash repro).
 *   - There is NO force-OFF. This guards against model codename leaks –
if
 *     we're not confident we're in an internal repo, we stay
undercover.
 *
 * All code paths are gated on process.env.USER_TYPE === 'ant'. Since
USER_TYPE is
 * a build-time --define, the bundler constant-folds these checks and
dead-code-
 * eliminates the ant-only branches from external builds. In external
builds every
 * function in this file reduces to a trivial return.
 */

import { getRepoClassCached } from './commitAttribution.js'
import { getGlobalConfig } from './config.js'

```

```

import { isEnvTruthy } from './envUtils.js'

export function isUndercover(): boolean {
  if (process.env.USER_TYPE === 'ant') {
    if (isEnvTruthy(process.env.CLAUDE_CODE_UNDERCOVER)) return true
    // Auto: active unless we've positively confirmed we're in an
    allowlisted
    // internal repo. 'external', 'none', and null (check not yet run)
    all
    // resolve to ON. The check is primed in setup.ts; only 'internal' →
    OFF.
    return getRepoClassCached() !== 'internal'
  }
  return false
}

export function getUndercoverInstructions(): string {
  if (process.env.USER_TYPE === 'ant') {
    return `### UNDERCOVER MODE – CRITICAL

```

You are operating UNDERCOVER in a PUBLIC/OPEN-SOURCE repository. Your commit messages, PR titles, and PR bodies MUST NOT contain ANY Anthropic-internal information. Do not blow your cover.

NEVER include in commit messages or PR descriptions:

- Internal model codenames (animal names like Capybara, Tengu, etc.)
- Unreleased model version numbers (e.g., opus-4-7, sonnet-4-8)
- Internal repo or project names (e.g., claude-cli-internal, anthropics/...)
- Internal tooling, Slack channels, or short links (e.g., go/cc, #claude-code-...)
- The phrase "Claude Code" or any mention that you are an AI
- Any hint of what model or version you are
- Co-Authored-By lines or any other attribution

Write commit messages as a human developer would – describe only what the code change does.

GOOD:

- "Fix race condition in file watcher initialization"
- "Add support for custom key bindings"
- "Refactor parser for better error messages"

BAD (never write these):

- "Fix bug found while testing with Claude Capybara"
- "1-shotted by claude-opus-4-6"
- "Generated with Claude Code"
- "Co-Authored-By: Claude Opus 4.6 <...>"

```
`  
    }  
    return ''  
}  
  
/**  
 * Check whether to show the one-time explainer dialog for auto-  
undercover.  
 * True when: undercover is active via auto-detection (not forced via  
env),  
 * and the user hasn't seen the notice before. Pure – the component  
marks the  
 * flag on mount.  
 */  
export function shouldShowUndercoverAutoNotice(): boolean {  
  if (process.env.USER_TYPE === 'ant') {  
    // If forced via env, user already knows; don't nag.  
    if (isEnvTruthy(process.env.CLAUDE_CODE_UNDERCOVER)) return false  
    if (!isUndercover()) return false  
    if (getGlobalConfig().hasSeenUndercoverAutoNotice) return false  
    return true  
  }  
}
```

```
return false  
}
```

5. Systempromptsections

SYSTEM PROMPT SECTIONS: The building blocks of Claude Code's system prompt. This reveals what instructions Anthropic injects into every conversation — the rules, constraints, and behavioral guidelines that shape how Claude Code behaves. Understanding this shows how Anthropic controls the agent's personality, safety boundaries, and operational rules.

```

import {
  clearBetaHeaderLatches,
  clearSystemPromptSectionState,
  getSystemPromptSectionCache,
  setSystemPromptSectionCacheEntry,
} from '../bootstrap/state.js'

type ComputeFn = () => string | null | Promise<string | null>

type SystemPromptSection = {
  name: string
  compute: ComputeFn
  cacheBreak: boolean
}

/**
 * Create a memoized system prompt section.
 * Computed once, cached until /clear or /compact.
 */
export function systemPromptSection(
  name: string,
  compute: ComputeFn,
): SystemPromptSection {
  return { name, compute, cacheBreak: false }
}

/**
 * Create a volatile system prompt section that recomputes every turn.
 * This WILL break the prompt cache when the value changes.
 * Requires a reason explaining why cache-breaking is necessary.
 */
export function DANGEROUS_uncachedSystemPromptSection(
  name: string,
  compute: ComputeFn,
  _reason: string,
): SystemPromptSection {
  return { name, compute, cacheBreak: true }
}

```

```

}

/**
 * Resolve all system prompt sections, returning prompt strings.
 */
export async function resolveSystemPromptSections(
  sections: SystemPromptSection[],
): Promise<(string | null)[]> {
  const cache = getSystemPromptSectionCache()

  return Promise.all(
    sections.map(async s => {
      if (!s.cacheBreak && cache.has(s.name)) {
        return cache.get(s.name) ?? null
      }
      const value = await s.compute()
      setSystemPromptSectionCacheEntry(s.name, value)
      return value
    }),
  )
}

/**
 * Clear all system prompt section state. Called on /clear and /compact.
 * Also resets beta header latches so a fresh conversation gets fresh
 * evaluation of AFK/fast-mode/cache-editing headers.
 */
export function clearSystemPromptSections(): void {
  clearSystemPromptSectionState()
  clearBetaHeaderLatches()
}

```

6. Tools Constants

TOOL CONSTANTS: The complete list and configuration of all 40+ tools Claude Code exposes. This shows the full capability surface and how tools are categorized. Key tools include: `FileRead`, `FileEdit`, `FileWrite`, `BashTool`, `GlobTool`, `GrepTool`, `WebFetchTool`, `WebSearchTool`, `AgentTool` (sub-agents), `NotebookEditTool`, and more specialized tools like `ScheduleCronTool` and `RemoteTriggerTool`.

```
// biome-ignore-all assist/source/organizeImports: ANT-ONLY import
markers must not be reordered
import { feature } from 'bun:bundle'
import { TASK_OUTPUT_TOOL_NAME } from
'../tools/TaskOutputTool/constants.js'
import { EXIT_PLAN_MODE_V2_TOOL_NAME } from
'../tools/ExitPlanModeTool/constants.js'
import { ENTER_PLAN_MODE_TOOL_NAME } from
'../tools/EnterPlanModeTool/constants.js'
import { AGENT_TOOL_NAME } from '../tools/AgentTool/constants.js'
import { ASK_USER_QUESTION_TOOL_NAME } from
'../tools/AskUserQuestionTool/prompt.js'
import { TASK_STOP_TOOL_NAME } from '../tools/TaskStopTool/prompt.js'
import { FILE_READ_TOOL_NAME } from '../tools/FileReadTool/prompt.js'
import { WEB_SEARCH_TOOL_NAME } from '../tools/WebSearchTool/prompt.js'
import { TODO_WRITE_TOOL_NAME } from
'../tools/ToDoWriteTool/constants.js'
import { GREP_TOOL_NAME } from '../tools/GrepTool/prompt.js'
import { WEB_FETCH_TOOL_NAME } from '../tools/WebFetchTool/prompt.js'
import { GLOB_TOOL_NAME } from '../tools/GlobTool/prompt.js'
import { SHELL_TOOL_NAMES } from '../utils/shell/shellToolUtils.js'
import { FILE_EDIT_TOOL_NAME } from '../tools/FileEditTool/constants.js'
import { FILE_WRITE_TOOL_NAME } from '../tools/FileWriteTool/prompt.js'
import { NOTEBOOK_EDIT_TOOL_NAME } from
'../tools/NotebookEditTool/constants.js'
import { SKILL_TOOL_NAME } from '../tools/SkillTool/constants.js'
import { SEND_MESSAGE_TOOL_NAME } from
'../tools/SendMessageTool/constants.js'
import { TASK_CREATE_TOOL_NAME } from
'../tools/TaskCreateTool/constants.js'
import { TASK_GET_TOOL_NAME } from '../tools/TaskGetTool/constants.js'
import { TASK_LIST_TOOL_NAME } from '../tools/TaskListTool/constants.js'
import { TASK_UPDATE_TOOL_NAME } from
'../tools/TaskUpdateTool/constants.js'
import { TOOL_SEARCH_TOOL_NAME } from
'../tools/ToolSearchTool/prompt.js'
import { SYNTHETIC_OUTPUT_TOOL_NAME } from
```

```
'../tools/SyntheticOutputTool/SyntheticOutputTool.js'
import { ENTER_WORKTREE_TOOL_NAME } from
'../tools/EnterWorktreeTool/constants.js'
import { EXIT_WORKTREE_TOOL_NAME } from
'../tools/ExitWorktreeTool/constants.js'
import { WORKFLOW_TOOL_NAME } from '../tools/WorkflowTool/constants.js'
import {
  CRON_CREATE_TOOL_NAME,
  CRON_DELETE_TOOL_NAME,
  CRON_LIST_TOOL_NAME,
} from '../tools/ScheduleCronTool/prompt.js'

export const ALL_AGENT_DISALLOWED_TOOLS = new Set([
  TASK_OUTPUT_TOOL_NAME,
  EXIT_PLAN_MODE_V2_TOOL_NAME,
  ENTER_PLAN_MODE_TOOL_NAME,
  // Allow Agent tool for agents when user is ant (enables nested
  agents)
  ...(process.env.USER_TYPE === 'ant' ? [] : [AGENT_TOOL_NAME]),
  ASK_USER_QUESTION_TOOL_NAME,
  TASK_STOP_TOOL_NAME,
  // Prevent recursive workflow execution inside subagents.
  ...(feature('WORKFLOW_SCRIPTS') ? [WORKFLOW_TOOL_NAME] : []),
])

export const CUSTOM_AGENT_DISALLOWED_TOOLS = new Set([
  ...ALL_AGENT_DISALLOWED_TOOLS,
])

/*
 * Async Agent Tool Availability Status (Source of Truth)
 */
export const ASYNC_AGENT_ALLOWED_TOOLS = new Set([
  FILE_READ_TOOL_NAME,
  WEB_SEARCH_TOOL_NAME,
  TODO_WRITE_TOOL_NAME,
  GREP_TOOL_NAME,
```

```
WEB_FETCH_TOOL_NAME,
GLOB_TOOL_NAME,
...SHELL_TOOL_NAMES,
FILE_EDIT_TOOL_NAME,
FILE_WRITE_TOOL_NAME,
NOTEBOOK_EDIT_TOOL_NAME,
SKILL_TOOL_NAME,
SYNTHETIC_OUTPUT_TOOL_NAME,
TOOL_SEARCH_TOOL_NAME,
ENTER_WORKTREE_TOOL_NAME,
EXIT_WORKTREE_TOOL_NAME,
])
/**
 * Tools allowed only for in-process teammates (not general async
agents).
 * These are injected by inProcessRunner.ts and allowed through
filterToolsForAgent
 * via isInProcessTeammate() check.
 */
export const IN_PROCESS_TEAMMATE_ALLOWED_TOOLS = new Set([
  TASK_CREATE_TOOL_NAME,
  TASK_GET_TOOL_NAME,
  TASK_LIST_TOOL_NAME,
  TASK_UPDATE_TOOL_NAME,
  SEND_MESSAGE_TOOL_NAME,
  // Teammate-created crons are tagged with the creating agentId and
routed to
  // that teammate's pendingUserMessages queue (see
useScheduledTasks.ts).
  ...(feature('AGENT_TRIGGERS')
    ? [CRON_CREATE_TOOL_NAME, CRON_DELETE_TOOL_NAME,
CRON_LIST_TOOL_NAME]
    : []),
])

/**
 * BLOCKED FOR ASYNC AGENTS:
```

```

* - AgentTool: Blocked to prevent recursion
* - TaskOutputTool: Blocked to prevent recursion
* - ExitPlanModeTool: Plan mode is a main thread abstraction.
* - TaskStopTool: Requires access to main thread task state.
* - TungstenTool: Uses singleton virtual terminal abstraction that
conflicts between agents.
*
* ENABLE LATER (NEED WORK):
* - MCPTool: TBD
* - ListMcpResourcesTool: TBD
* - ReadMcpResourceTool: TBD
*/

/**
 * Tools allowed in coordinator mode - only output and agent management
tools for the coordinator
 */
export const COORDINATOR_MODE_ALLOWED_TOOLS = new Set([
  AGENT_TOOL_NAME,
  TASK_STOP_TOOL_NAME,
  SEND_MESSAGE_TOOL_NAME,
  SYNTHETIC_OUTPUT_TOOL_NAME,
])

```

7. Fastmode

PENGUIN MODE (Fast Mode): The internal codename for what users know as "Fast Mode." The API endpoint is `/api/claude_code_penguin_mode` and it has a kill-switch flag called `tengu_penguins_off`. This file reveals how Anthropic implements the faster, lighter model — it uses a smaller/cheaper model variant for quick interactions while preserving the full model for complex tasks. The "penguin" naming follows their internal convention of using animal codenames.

```
import axios from 'axios'
import { getOAuthConfig, OAUTH_BETA_HEADER } from
'src/constants/oauth.js'
import { getFeatureValue_CACHED_MAY_BE_STALE } from
'src/services/analytics/growthbook.js'
import {
  getIsNonInteractiveSession,
  getKairosActive,
  preferThirdPartyAuthentication,
} from '../bootstrap/state.js'
import {
  type AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
  logEvent,
} from '../services/analytics/index.js'
import {
  getAnthropicApiKey,
  getClaudeAIOAuthTokens,
  handleOAuth401Error,
  hasProfileScope,
} from './auth.js'
import { isInBundledMode } from './bundledMode.js'
import { getGlobalConfig, saveGlobalConfig } from './config.js'
import { logForDebugging } from './debug.js'
import { isEnvTruthy } from './envUtils.js'
import {
  getDefaultMainLoopModelSetting,
  isOpus1mMergeEnabled,
  type ModelSetting,
  parseUserSpecifiedModel,
} from './model/model.js'
import { getAPIProvider } from './model/providers.js'
import { isEssentialTrafficOnly } from './privacyLevel.js'
import {
  getInitialSettings,
  getSettingsForSource,
  updateSettingsForSource,
} from './settings/settings.js'
```

```
import { createSignal } from './signal.js'

export function isFastModeEnabled(): boolean {
  return !isEnvTruthy(process.env.CLAUDE_CODE_DISABLE_FAST_MODE)
}

export function isFastModeAvailable(): boolean {
  if (!isFastModeEnabled()) {
    return false
  }
  return getFastModeUnavailableReason() === null
}

type AuthType = 'oauth' | 'api-key'

function getDisabledReasonMessage(
  disabledReason: FastModeDisabledReason,
  authType: AuthType,
): string {
  switch (disabledReason) {
    case 'free':
      return authType === 'oauth'
        ? 'Fast mode requires a paid subscription'
        : 'Fast mode unavailable during evaluation. Please purchase credits.'
    case 'preference':
      return 'Fast mode has been disabled by your organization'
    case 'extra_usage_disabled':
      // Only OAuth users can have extra_usage_disabled; console users
      // don't have this concept
      return 'Fast mode requires extra usage billing · /extra-usage to enable'
    case 'network_error':
      return 'Fast mode unavailable due to network connectivity issues'
    case 'unknown':
      return 'Fast mode is currently unavailable'
  }
}
```

```

}

export function getFastModeUnavailableReason(): string | null {
  if (!isFastModeEnabled()) {
    return 'Fast mode is not available'
  }

  const statigReason = getFeatureValue_CACHED_MAY_BE_STALE(
    'tengu_penguins_off',
    null,
  )
  // Statsig reason has priority over other reasons.
  if (statigReason !== null) {
    logForDebugging(`Fast mode unavailable: ${statigReason}`)
    return statigReason
  }

  // Previously, fast mode required the native binary (bun build). This
  is no
  // longer necessary, but we keep this option behind a flag just in
  case.
  if (
    !isInBundledMode() &&
    getFeatureValue_CACHED_MAY_BE_STALE('tengu_marble_sandcastle',
false)
  ) {
    return 'Fast mode requires the native binary · Install from:
https://claude.com/product/claude-code'
  }

  // Not available in the SDK unless explicitly opted in via --settings.
  // Assistant daemon mode is exempt – it's first-party orchestration,
  and
  // kairosActive is set before this check runs (main.tsx:~1626 vs
  ~3249).
  if (
    getIsNonInteractiveSession() &&

```

```

    preferThirdPartyAuthentication() &&
    !getKairosActive()
  ) {
    const flagFastMode = getSettingsForSource('flagSettings')?.fastMode
    if (!flagFastMode) {
      const reason = 'Fast mode is not available in the Agent SDK'
      logForDebugging(`Fast mode unavailable: ${reason}`)
      return reason
    }
  }

  // Only available for 1P (not Bedrock/Vertex/Foundry)
  if (getAPIProvider() !== 'firstParty') {
    const reason = 'Fast mode is not available on Bedrock, Vertex, or
Foundry'
    logForDebugging(`Fast mode unavailable: ${reason}`)
    return reason
  }

  if (orgStatus.status === 'disabled') {
    if (
      orgStatus.reason === 'network_error' ||
      orgStatus.reason === 'unknown'
    ) {
      // The org check can fail behind corporate proxies that block the
      // endpoint. We add CLAUDE_CODE_SKIP_FAST_MODE_NETWORK_ERRORS=1 to
      // bypass this check in the CC binary. This is OK since we have
      // another check in the API to error out when disabled by org.
      if
(isEnvTruthy(process.env.CLAUDE_CODE_SKIP_FAST_MODE_NETWORK_ERRORS)) {
        return null
      }
    }
    const authType: AuthType =
      getClaudeAIOAuthTokens() !== null ? 'oauth' : 'api-key'
    const reason = getDisabledReasonMessage(orgStatus.reason, authType)
    logForDebugging(`Fast mode unavailable: ${reason}`)
  }

```

```

    return reason
  }

  return null
}

// @[MODEL LAUNCH]: Update supported Fast Mode models.
export const FAST_MODE_MODEL_DISPLAY = 'Opus 4.6'

export function getFastModeModel(): string {
  return 'opus' + (isOpus1mMergeEnabled() ? '[1m]' : '')
}

export function getInitialFastModeSetting(model: ModelSetting): boolean
{
  if (!isFastModeEnabled()) {
    return false
  }
  if (!isFastModeAvailable()) {
    return false
  }
  if (!isFastModeSupportedByModel(model)) {
    return false
  }
  const settings = getInitialSettings()
  // If per-session opt-in is required, fast mode starts off each
session
  if (settings.fastModePerSessionOptIn) {
    return false
  }
  return settings.fastMode === true
}

export function isFastModeSupportedByModel(
  modelSetting: ModelSetting,
): boolean {
  if (!isFastModeEnabled()) {

```

```

    return false
  }
  const model = modelSetting ?? getDefaultMainLoopModelSetting()
  const parsedModel = parseUserSpecifiedModel(model)
  return parsedModel.toLowerCase().includes('opus-4-6')
}

// --- Fast mode runtime state ---
// Separate from user preference (settings.fastMode). This tracks the
// actual
// operational state: whether we're actively sending fast speed or in
// cooldown
// after a rate limit.

export type FastModeRuntimeState =
  | { status: 'active' }
  | { status: 'cooldown'; resetAt: number; reason: CooldownReason }

let runtimeState: FastModeRuntimeState = { status: 'active' }
let hasLoggedCooldownExpiry = false

// --- Cooldown event listeners ---
export type CooldownReason = 'rate_limit' | 'overloaded'

const cooldownTriggered =
  createSignal<[resetAt: number, reason: CooldownReason]>()
const cooldownExpired = createSignal()
export const onCooldownTriggered = cooldownTriggered.subscribe
export const onCooldownExpired = cooldownExpired.subscribe

export function getFastModeRuntimeState(): FastModeRuntimeState {
  if (
    runtimeState.status === 'cooldown' &&
    Date.now() >= runtimeState.resetAt
  ) {
    if (isFastModeEnabled() && !hasLoggedCooldownExpiry) {
      logForDebugging('Fast mode cooldown expired, re-enabling fast

```

```

mode')
    hasLoggedCooldownExpiry = true
    cooldownExpired.emit()
  }
  runtimeState = { status: 'active' }
}
return runtimeState
}

export function triggerFastModeCooldown(
  resetTimestamp: number,
  reason: CooldownReason,
): void {
  if (!isFastModeEnabled()) {
    return
  }
  runtimeState = { status: 'cooldown', resetAt: resetTimestamp, reason }
  hasLoggedCooldownExpiry = false
  const cooldownDurationMs = resetTimestamp - Date.now()
  logForDebugging(
    `Fast mode cooldown triggered (${reason}), duration
    ${Math.round(cooldownDurationMs / 1000)}s`,
  )
  logEvent('tengu_fast_mode_fallback_triggered', {
    cooldown_duration_ms: cooldownDurationMs,
    cooldown_reason:
      reason as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
  })
  cooldownTriggered.emit(resetTimestamp, reason)
}

export function clearFastModeCooldown(): void {
  runtimeState = { status: 'active' }
}

/**

```

```

* Called when the API rejects a fast mode request (e.g., 400 "Fast mode
is
* not enabled for your organization"). Permanently disables fast mode
using
* the same flow as when the prefetch discovers the org has it disabled.
*/
export function handleFastModeRejectedByAPI(): void {
  if (orgStatus.status === 'disabled') {
    return
  }
  orgStatus = { status: 'disabled', reason: 'preference' }
  updateSettingsForSource('userSettings', { fastMode: undefined })
  saveGlobalConfig(current => ({
    ...current,
    penguinModeOrgEnabled: false,
  }))
  orgFastModeChange.emit(false)
}

// --- Overage rejection listeners ---
// Fired when a 429 indicates fast mode was rejected because extra usage
// (overage billing) is not available. Distinct from org-level
// disabling.
const overageRejection = createSignal<[message: string]>()
export const onFastModeOverageRejection = overageRejection.subscribe

function getOverageDisabledMessage(reason: string | null): string {
  switch (reason) {
    case 'out_of_credits':
      return 'Fast mode disabled · extra usage credits exhausted'
    case 'org_level_disabled':
    case 'org_service_level_disabled':
      return 'Fast mode disabled · extra usage disabled by your
organization'
    case 'org_level_disabled_until':
      return 'Fast mode disabled · extra usage spending cap reached'
    case 'member_level_disabled':

```

```

        return 'Fast mode disabled · extra usage disabled for your
account'
    case 'seat_tier_level_disabled':
    case 'seat_tier_zero_credit_limit':
    case 'member_zero_credit_limit':
        return 'Fast mode disabled · extra usage not available for your
plan'
    case 'overage_not_provisioned':
    case 'no_limits_configured':
        return 'Fast mode requires extra usage billing · /extra-usage to
enable'
    default:
        return 'Fast mode disabled · extra usage not available'
    }
}

function isOutOfCreditsReason(reason: string | null): boolean {
    return reason === 'org_level_disabled_until' || reason ===
'out_of_credits'
}

/**
 * Called when a 429 indicates fast mode was rejected because extra
usage
 * is not available. Permanently disables fast mode (unless the user has
 * ran out of credits) and notifies with a reason-specific message.
 */
export function handleFastModeOverageRejection(reason: string | null):
void {
    const message = getOverageDisabledMessage(reason)
    logForDebugging(
        `Fast mode overage rejection: ${reason ?? 'unknown'} – ${message}`,
    )
    logEvent('tengu_fast_mode_overage_rejected', {
        overage_disabled_reason: (reason ??
            'unknown') as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,

```

```

    })
    // Disable fast mode permanently unless the user has ran out of
credits
    if (!isOutOfCreditsReason(reason)) {
        updateSettingsForSource('userSettings', { fastMode: undefined })
        saveGlobalConfig(current => ({
            ...current,
            penguinModeOrgEnabled: false,
        }))
    }
    overageRejection.emit(message)
}

export function isFastModeCooldown(): boolean {
    return getFastModeRuntimeState().status === 'cooldown'
}

export function getFastModeState(
    model: ModelSetting,
    fastModeUserEnabled: boolean | undefined,
): 'off' | 'cooldown' | 'on' {
    const enabled =
        isFastModeEnabled() &&
        isFastModeAvailable() &&
        !!fastModeUserEnabled &&
        isFastModeSupportedByModel(model)
    if (enabled && isFastModeCooldown()) {
        return 'cooldown'
    }
    if (enabled) {
        return 'on'
    }
    return 'off'
}

// Disabled reason returned by the API. The API is the canonical source
for why

```

```

// fast mode is disabled (free account, admin preference, extra usage
not enabled).
export type FastModeDisabledReason =
  | 'free'
  | 'preference'
  | 'extra_usage_disabled'
  | 'network_error'
  | 'unknown'

// In-memory cache of the fast mode status from the API.
// Distinct from the user's fastMode app state – this represents
// whether the org *allows* fast mode and why it may be disabled.
// Modeled as a discriminated union so the invalid state
// (disabled without a reason) is unrepresentable.
type FastModeOrgStatus =
  | { status: 'pending' }
  | { status: 'enabled' }
  | { status: 'disabled'; reason: FastModeDisabledReason }

let orgStatus: FastModeOrgStatus = { status: 'pending' }

// Listeners notified when org-level fast mode status changes
const orgFastModeChange = createSignal<[orgEnabled: boolean]>()
export const onOrgFastModeChanged = orgFastModeChange.subscribe

type FastModeResponse = {
  enabled: boolean
  disabled_reason: FastModeDisabledReason | null
}

async function fetchFastModeStatus(
  auth: { accessToken: string } | { apiKey: *** },
): Promise<FastModeResponse> {
  const endpoint =
`${getOAuthConfig().BASE_API_URL}/api/claude_code_penguin_mode`
  const headers: Record<string, string> =
    'accessToken' in auth

```

```

    ? {
      Authorization: `*** ${auth.accessToken}`,
      'anthropic-beta': OAUTH_BETA_HEADER,
    }
    : { 'x-api-key': auth.apiKey }

    const response = await axios.get<FastModeResponse>(endpoint, { headers
  })
  return response.data
}

const PREFETCH_MIN_INTERVAL_MS = 30_000
let lastPrefetchAt = 0
let inflightPrefetch: Promise<void> | null = null

/**
 * Resolve orgStatus from the persisted cache without making any API
calls.
 * Used when startup prefetches are throttled to avoid hitting the
network
 * while still making fast mode availability checks work.
 */
export function resolveFastModeStatusFromCache(): void {
  if (!isFastModeEnabled()) {
    return
  }
  if (orgStatus.status !== 'pending') {
    return
  }
  const isAnt = process.env.USER_TYPE === 'ant'
  const cachedEnabled = getGlobalConfig().penguinModeOrgEnabled === true
  orgStatus =
    isAnt || cachedEnabled
      ? { status: 'enabled' }
      : { status: 'disabled', reason: 'unknown' }
}

```

```

export async function prefetchFastModeStatus(): Promise<void> {
  // Skip network requests if nonessential traffic is disabled
  if (isEssentialTrafficOnly()) {
    return
  }

  if (!isFastModeEnabled()) {
    return
  }

  if (inflightPrefetch) {
    logForDebugging(
      'Fast mode prefetch in progress, returning in-flight promise',
    )
    return inflightPrefetch
  }

  // Service key OAuth sessions lack user:profile scope → endpoint 403s.
  // Resolve orgStatus from cache and bail before burning the throttle
  window.

  // API key auth is unaffected.
  const apiKey = getAnthropicApiKey()
  const hasUsableOAuth =
    getClaudeAIOAuthTokens()?.accessToken && hasProfileScope()
  if (!hasUsableOAuth && !apiKey) {
    const isAnt = process.env.USER_TYPE === 'ant'
    const cachedEnabled = getGlobalConfig().penguinModeOrgEnabled ===
true
    orgStatus =
      isAnt || cachedEnabled
        ? { status: 'enabled' }
        : { status: 'disabled', reason: 'preference' }
    return
  }

  const now = Date.now()
  if (now - lastPrefetchAt < PREFETCH_MIN_INTERVAL_MS) {

```

```

    logForDebugging('Skipping fast mode prefetch, fetched recently')
    return
  }
  lastPrefetchAt = now

const fetchWithCurrentAuth = async (): Promise<FastModeResponse> => {
  const currentTokens = getClaudeAIOAuthTokens()
  const auth =
    currentTokens?.accessToken && hasProfileScope()
      ? { accessToken: currentTokens.accessToken }
      : apiKey
      ? { apiKey }
      : null
  if (!auth) {
    throw new Error('No auth available')
  }
  return fetchFastModeStatus(auth)
}

async function doFetch(): Promise<void> {
  try {
    let status: FastModeResponse
    try {
      status = await fetchWithCurrentAuth()
    } catch (err) {
      const isAuthError =
        axios.isAxiosError(err) &&
        (err.response?.status === 401 ||
          (err.response?.status === 403 &&
            typeof err.response?.data === 'string' &&
            err.response.data.includes('OAuth token has been
revoked'))))
      if (isAuthError) {
        const failedAccessToken =
getClaudeAIOAuthTokens()?.accessToken
        if (failedAccessToken) {
          await handleOAuth401Error(failedAccessToken)

```

```

        status = await fetchWithCurrentAuth()
      } else {
        throw err
      }
    } else {
      throw err
    }
  }

const previousEnabled =
  orgStatus.status !== 'pending'
    ? orgStatus.status === 'enabled'
    : getGlobalConfig().penguinModeOrgEnabled
orgStatus = status.enabled
  ? { status: 'enabled' }
  : {
    status: 'disabled',
    reason: status.disabled_reason ?? 'preference',
  }

if (previousEnabled !== status.enabled) {
  // When org disables fast mode, permanently turn off the user's
fast mode setting
  if (!status.enabled) {
    updateSettingsForSource('userSettings', { fastMode: undefined
})
  }
  saveGlobalConfig(current => ({
    ...current,
    penguinModeOrgEnabled: status.enabled,
  }))
  orgFastModeChange.emit(status.enabled)
}

logForDebugging(
  `Org fast mode: ${status.enabled ? 'enabled' : 'disabled'
} (${status.disabled_reason ?? 'preference'})`,
)
} catch (err) {

```

```

        // On failure: ants default to enabled (don't block internal
users).
        // External users: fall back to the cached penguinModeOrgEnabled
value;
        // if no positive cache, disable with network_error reason.
const isAnt = process.env.USER_TYPE === 'ant'
const cachedEnabled = getGlobalConfig().penguinModeOrgEnabled ===
true
orgStatus =
    isAnt || cachedEnabled
        ? { status: 'enabled' }
        : { status: 'disabled', reason: 'network_error' }
logForDebugging(
    `Failed to fetch org fast mode status, defaulting to
${orgStatus.status === 'enabled' ? 'enabled (cached)' : 'disabled
(network_error)'}: ${err}`,
    { level: 'error' },
)
logEvent('tengu_org_penguin_mode_fetch_failed', {})
} finally {
    inflightPrefetch = null
}
}

inflightPrefetch = doFetch()
return inflightPrefetch
}

```

8. Autodream

AUTO-DREAM: When Claude literally "dreams" during idle time. This memory consolidation system has a three-gate trigger: (1) At least 24 hours since the last dream cycle, (2) At least 5 sessions completed since last consolidation, (3) A consolidation lock preventing concurrent processes. When all three conditions are met, the dream cycle runs four phases: Orient (assess current memory state), Gather Recent Signal (collect insights from recent sessions), Consolidate (merge new knowledge with existing memory), and Prune & Index (remove redundancies, keep MEMORY.md under 200 lines / ~25KB). This is similar to how human sleep consolidates memories.

```

// biome-ignore-all assist/source/organizeImports: ANT-ONLY import
markers must not be reordered
// Background memory consolidation. Fires the /dream prompt as a forked
// subagent when time-gate passes AND enough sessions have accumulated.
//
// Gate order (cheapest first):
//   1. Time: hours since lastConsolidatedAt >= minHours (one stat)
//   2. Sessions: transcript count with mtime > lastConsolidatedAt >=
minSessions
//   3. Lock: no other process mid-consolidation
//
// State is closure-scoped inside initAutoDream() rather than module-
level
// (tests call initAutoDream() in beforeEach for a fresh closure).

import type { REPLHookContext } from
'../../utils/hooks/postSamplingHooks.js'
import {
  createCacheSafeParams,
  runForkedAgent,
} from '../../utils/forkedAgent.js'
import {
  createUserMessage,
  createMemorySavedMessage,
} from '../../utils/messages.js'
import type { Message } from '../../types/message.js'
import { logForDebugging } from '../../utils/debug.js'
import type { ToolUseContext } from '../../Tool.js'
import { logEvent } from '../../analytics/index.js'
import { getFeatureValue_CACHED_MAY_BE_STALE } from
'../../analytics/growthbook.js'
import { isAutoMemoryEnabled, getAutoMemPath } from
'../../memdir/paths.js'
import { isAutoDreamEnabled } from './config.js'
import { getProjectDir } from '../../utils/sessionStorage.js'
import {
  getOriginalCwd,

```

```

    getKairosActive,
    getIsRemoteMode,
    getSessionId,
} from '../bootstrap/state.js'
import { createAutoMemCanUseTool } from
'../extractMemories/extractMemories.js'
import { buildConsolidationPrompt } from './consolidationPrompt.js'
import {
    readLastConsolidatedAt,
    listSessionsTouchedSince,
    tryAcquireConsolidationLock,
    rollbackConsolidationLock,
} from './consolidationLock.js'
import {
    registerDreamTask,
    addDreamTurn,
    completeDreamTask,
    failDreamTask,
    isDreamTask,
} from '../tasks/DreamTask/DreamTask.js'
import { FILE_EDIT_TOOL_NAME } from
'../tools/FileEditTool/constants.js'
import { FILE_WRITE_TOOL_NAME } from
'../tools/FileWriteTool/prompt.js'

// Scan throttle: when time-gate passes but session-gate doesn't, the
lock
// mtime doesn't advance, so the time-gate keeps passing every turn.
const SESSION_SCAN_INTERVAL_MS = 10 * 60 * 1000

type AutoDreamConfig = {
    minHours: number
    minSessions: number
}

const DEFAULTS: AutoDreamConfig = {
    minHours: 24,

```

```

    minSessions: 5,
}

/**
 * Thresholds from tengu_onyx_plover. The enabled gate lives in
config.ts
 * (isAutoDreamEnabled); this returns only the scheduling knobs.
Defensive
 * per-field validation since GB cache can return stale wrong-type
values.
 */
function getConfig(): AutoDreamConfig {
    const raw =
        getFeatureValue_CACHED_MAY_BE_STALE<Partial<AutoDreamConfig> | null>
(
    'tengu_onyx_plover',
    null,
)
    return {
        minHours:
            typeof raw?.minHours === 'number' &&
            Number.isFinite(raw.minHours) &&
            raw.minHours > 0
                ? raw.minHours
                : DEFAULTS.minHours,
        minSessions:
            typeof raw?.minSessions === 'number' &&
            Number.isFinite(raw.minSessions) &&
            raw.minSessions > 0
                ? raw.minSessions
                : DEFAULTS.minSessions,
    }
}

function isGateOpen(): boolean {
    if (getKairosActive()) return false // KAIROS mode uses disk-skill
dream

```

```

    if (getIsRemoteMode()) return false
    if (!isAutoMemoryEnabled()) return false
    return isAutoDreamEnabled()
}

// Ant-build-only test override. Bypasses enabled/time/session gates but
// NOT
// the lock (so repeated turns don't pile up dreams) or the memory-dir
// precondition. Still scans sessions so the prompt's session-hint is
// populated.
function isForced(): boolean {
    return false
}

type AppendSystemMessageFn =
Nonnullable<ToolUseContext['appendSystemMessage']>

let runner:
  | ((
    context: REPLHookContext,
    appendSystemMessage?: AppendSystemMessageFn,
  ) => Promise<void>)
  | null = null

/**
 * Call once at startup (from backgroundHousekeeping alongside
 * initExtractMemories), or per-test in beforeEach for a fresh closure.
 */
export function initAutoDream(): void {
    let lastSessionScanAt = 0

    runner = async function runAutoDream(context, appendSystemMessage) {
        const cfg = getConfig()
        const force = isForced()
        if (!force && !isGateOpen()) return

        // --- Time gate ---

```

```

let lastAt: number
try {
  lastAt = await readLastConsolidatedAt()
} catch (e: unknown) {
  logForDebugging(
    `[autoDream] readLastConsolidatedAt failed: ${e as
Error).message}`,
  )
  return
}
const hoursSince = (Date.now() - lastAt) / 3_600_000
if (!force && hoursSince < cfg.minHours) return

// --- Scan throttle ---
const sinceScanMs = Date.now() - lastSessionScanAt
if (!force && sinceScanMs < SESSION_SCAN_INTERVAL_MS) {
  logForDebugging(
    `[autoDream] scan throttle – time-gate passed but last scan was
${Math.round(sinceScanMs / 1000)}s ago`,
  )
  return
}
lastSessionScanAt = Date.now()

// --- Session gate ---
let sessionIds: string[]
try {
  sessionIds = await listSessionsTouchedSince(lastAt)
} catch (e: unknown) {
  logForDebugging(
    `[autoDream] listSessionsTouchedSince failed: ${e as
Error).message}`,
  )
  return
}
// Exclude the current session (its mtime is always recent).
const currentSession = getSessionId()

```

```

    sessionIds = sessionIds.filter(id => id !== currentSession)
    if (!force && sessionIds.length < cfg.minSessions) {
      logForDebugging(
        `[autoDream] skip – ${sessionIds.length} sessions since last
consolidation, need ${cfg.minSessions}`,
      )
      return
    }

    // --- Lock ---
    // Under force, skip acquire entirely – use the existing mtime so
    // kill's rollback is a no-op (rewinds to where it already is).
    // The lock file stays untouched; next non-force turn sees it as-is.
    let priorMtime: number | null
    if (force) {
      priorMtime = lastAt
    } else {
      try {
        priorMtime = await tryAcquireConsolidationLock()
      } catch (e: unknown) {
        logForDebugging(
          `[autoDream] lock acquire failed: ${(e as Error).message}`,
        )
        return
      }
      if (priorMtime === null) return
    }

    logForDebugging(
      `[autoDream] firing – ${hoursSince.toFixed(1)}h since last,
${sessionIds.length} sessions to review`,
    )
    logEvent('tengu_auto_dream_fired', {
      hours_since: Math.round(hoursSince),
      sessions_since: sessionIds.length,
    })

```

```
const setAppState =
  context.toolUseContext.setAppStateForTasks ??
  context.toolUseContext.setAppState
const abortController = new AbortController()
const taskId = registerDreamTask(setAppState, {
  sessionsReviewing: sessionIds.length,
  priorMtime,
  abortController,
})
```

```
try {
  const memoryRoot = getAutoMemPath()
  const transcriptDir = getProjectDir(getOriginalCwd())
  // Tool constraints note goes in `extra`, not the shared prompt
```

body –

```
  // manual /dream runs in the main loop with normal permissions and
  this
  // would be misleading there.
  const extra = `
```

****Tool constraints for this run:**** Bash is restricted to read-only commands (`\`ls\``, `\`find\``, `\`grep\``, `\`cat\``, `\`stat\``, `\`wc\``, `\`head\``, `\`tail\``, and similar). Anything that writes, redirects to a file, or modifies state will be denied. Plan your exploration with this in mind – no need to probe.

Sessions since last consolidation (\${sessionIds.length}):

```
${sessionIds.map(id => `- ${id}`).join('\n')}`
```

```
  const prompt = buildConsolidationPrompt(memoryRoot, transcriptDir,
  extra)
```

```
const result = await runForkedAgent({
  promptMessages: [createUserMessage({ content: prompt })],
  cacheSafeParams: createCacheSafeParams(context),
  canUseTool: createAutoMemCanUseTool(memoryRoot),
  querySource: 'auto_dream',
  forkLabel: 'auto_dream',
```

```

        skipTranscript: true,
        overrides: { abortController },
        onMessage: makeDreamProgressWatcher(taskId, setAppState),
    })

    completeDreamTask(taskId, setAppState)
    // Inline completion summary in the main transcript (same surface
as
    // extractMemories's "Saved N memories" message).
    const dreamState = context.toolUseContext.getAppState().tasks?.
[taskId]
    if (
        appendSystemMessage &&
        isDreamTask(dreamState) &&
        dreamState.filesTouched.length > 0
    ) {
        appendSystemMessage({
            ...createMemorySavedMessage(dreamState.filesTouched),
            verb: 'Improved',
        })
    }
    logForDebugging(
        `[autoDream] completed – cache:
read=${result.totalUsage.cache_read_input_tokens}
created=${result.totalUsage.cache_creation_input_tokens}`,
    )
    logEvent('tengu_auto_dream_completed', {
        cache_read: result.totalUsage.cache_read_input_tokens,
        cache_created: result.totalUsage.cache_creation_input_tokens,
        output: result.totalUsage.output_tokens,
        sessions_reviewed: sessionIds.length,
    })
} catch (e: unknown) {
    // If the user killed from the bg-tasks dialog, DreamTask.kill
already
    // aborted, rolled back the lock, and set status=killed. Don't
overwrite

```

```

    // or double-rollback.
    if (abortController.signal.aborted) {
        logForDebugging('[autoDream] aborted by user')
        return
    }
    logForDebugging(`[autoDream] fork failed: ${e as
Error).message}`)
    logEvent('tengu_auto_dream_failed', {})
    failDreamTask(taskId, setAppState)
    // Rewind mtime so time-gate passes again. Scan throttle is the
backoff.
    await rollbackConsolidationLock(priorMtime)
}
}
}

/**
 * Watch the forked agent's messages. For each assistant turn, extracts
any
 * text blocks (the agent's reasoning/summary – what the user wants to
see)
 * and collapses tool_use blocks to a count. Edit/Write file_paths are
 * collected for phase-flip + the inline completion message.
 */
function makeDreamProgressWatcher(
    taskId: string,
    setAppState: import('../Task.js').SetAppState,
): (msg: Message) => void {
    return msg => {
        if (msg.type !== 'assistant') return
        let text = ''
        let toolUseCount = 0
        const touchedPaths: string[] = []
        for (const block of msg.message.content) {
            if (block.type === 'text') {
                text += block.text
            } else if (block.type === 'tool_use') {

```

```

        toolUseCount++
        if (
            block.name === FILE_EDIT_TOOL_NAME ||
            block.name === FILE_WRITE_TOOL_NAME
        ) {
            const input = block.input as { file_path?: unknown }
            if (typeof input.file_path === 'string') {
                touchedPaths.push(input.file_path)
            }
        }
    }
}
addDreamTurn(
    taskId,
    { text: text.trim(), toolUseCount },
    touchedPaths,
    setAppState,
)
}
}

/**
 * Entry point from stopHooks. No-op until initAutoDream() has been
 * called.
 * Per-turn cost when enabled: one GB cache read + one stat.
 */
export async function executeAutoDream(
    context: REPLHookContext,
    appendSystemMessage?: AppendSystemMessageFn,
): Promise<void> {
    await runner?.(context, appendSystemMessage)
}

```

9. Compact

CONTEXT COMPACTION: The 5 distinct compaction strategies Claude Code uses to manage context window limits. This is why Claude Code can handle long conversations without running out of tokens. The strategies include: reactive compression (triggered when approaching limits), micro-compression (small summarizations), trimmed compression (cutting less important parts), context collapsing (CONTEXT_COLLAPSE for emergency situations), and AI-powered summarization. The system also tracks 14 cache-break vectors — conditions that could invalidate prompt caching.

```

import { feature } from 'bun:bundle'
import type { UUID } from 'crypto'
import uniqBy from 'lodash-es/uniqBy.js'

/* eslint-disable @typescript-eslint/no-require-imports */
const sessionTranscriptModule = feature('KAIR0S')
  ? (require('../sessionTranscript/sessionTranscript.js') as typeof
import('../sessionTranscript/sessionTranscript.js'))
  : null

import { APIUserAbortError } from '@anthropic-ai/sdk'
import { markPostCompaction } from 'src/bootstrap/state.js'
import { getInvokedSkillsForAgent } from '../../bootstrap/state.js'
import type { QuerySource } from '../../constants/querySource.js'
import type { CanUseToolFn } from '../../hooks/useCanUseTool.js'
import type { Tool, ToolUseContext } from '../../Tool.js'
import type { LocalAgentTaskState } from
'../../tasks/LocalAgentTask/LocalAgentTask.js'
import { FileReadTool } from '../../tools/FileReadTool/FileReadTool.js'
import {
  FILE_READ_TOOL_NAME,
  FILE_UNCHANGED_STUB,
} from '../../tools/FileReadTool/prompt.js'
import { ToolSearchTool } from
'../../tools/ToolSearchTool/ToolSearchTool.js'
import type { AgentId } from '../../types/ids.js'
import type {
  AssistantMessage,
  AttachmentMessage,
  HookResultMessage,
  Message,
  PartialCompactDirection,
  SystemCompactBoundaryMessage,
  SystemMessage,
  UserMessage,
} from '../../types/message.js'
import {

```

```
    createAttachmentMessage,
    generateFileAttachment,
    getAgentListingDeltaAttachment,
    getDeferredToolsDeltaAttachment,
    getMcpInstructionsDeltaAttachment,
} from '../utils/attachments.js'
import { getMemoryPath } from '../utils/config.js'
import { COMPACT_MAX_OUTPUT_TOKENS } from '../utils/context.js'
import {
    analyzeContext,
    tokenStatsToStatsigMetrics,
} from '../utils/contextAnalysis.js'
import { logForDebugging } from '../utils/debug.js'
import { hasExactErrorMessage } from '../utils/errors.js'
import { cacheToObject } from '../utils/fileStateCache.js'
import {
    type CacheSafeParams,
    runForkedAgent,
} from '../utils/forkedAgent.js'
import {
    executePostCompactHooks,
    executePreCompactHooks,
} from '../utils/hooks.js'
import { logError } from '../utils/log.js'
import { MEMORY_TYPE_VALUES } from '../utils/memory/types.js'
import {
    createCompactBoundaryMessage,
    createUserMessage,
    getAssistantMessageText,
    getLastAssistantMessage,
    getMessagesAfterCompactBoundary,
    isCompactBoundaryMessage,
    normalizeMessagesForAPI,
} from '../utils/messages.js'
import { expandPath } from '../utils/path.js'
import { getPlan, getPlanFilePath } from '../utils/plans.js'
import {
```

```
    isSessionActivityTrackingActive,  
    sendSessionActivitySignal,  
} from '../..//utils/sessionActivity.js'  
import { processSessionStartHooks } from '../..//utils/sessionStart.js'  
import {  
    getTranscriptPath,  
    reAppendSessionMetadata,  
} from '../..//utils/sessionStorage.js'  
import { sleep } from '../..//utils/sleep.js'  
import { jsonStringify } from '../..//utils/slowOperations.js'  
/* eslint-enable @typescript-eslint/no-require-imports */  
import { asSystemPrompt } from '../..//utils/systemPromptType.js'  
import { getTaskOutputPath } from '../..//utils/task/diskOutput.js'  
import {  
    getTokenUsage,  
    tokenCountFromLastAPIResponse,  
    tokenCountWithEstimation,  
} from '../..//utils/tokens.js'  
import {  
    extractDiscoveredToolNames,  
    isToolSearchEnabled,  
} from '../..//utils/toolSearch.js'  
import { getFeatureValue_CACHED_MAY_BE_STALE } from  
'../analytics/growthbook.js'  
import {  
    type AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,  
    logEvent,  
} from '../analytics/index.js'  
import {  
    getMaxOutputTokensForModel,  
    queryModelWithStreaming,  
} from '../api/claude.js'  
import {  
    getPromptTooLongTokenGap,  
    PROMPT_TOO_LONG_ERROR_MESSAGE,  
    startsWithApiErrorPrefix,  
} from '../api/errors.js'
```

```
import { notifyCompaction } from '../api/promptCacheBreakDetection.js'
import { getRetryDelay } from '../api/withRetry.js'
import { logPermissionContextForAnts } from '../internalLogging.js'
import {
  roughTokenCountEstimation,
  roughTokenCountEstimationForMessages,
} from '../tokenEstimation.js'
import { groupMessagesByApiRound } from './grouping.js'
import {
  getCompactPrompt,
  getCompactUserSummaryMessage,
  getPartialCompactPrompt,
} from './prompt.js'

export const POST_COMPACT_MAX_FILES_TO_RESTORE = 5
export const POST_COMPACT_TOKEN_BUDGET = 50_000
export const POST_COMPACT_MAX_TOKENS_PER_FILE = 5_000
// Skills can be large (verify=18.7KB, claude-api=20.1KB). Previously
re-injected
// unbounded on every compact → 5-10K tok/compact. Per-skill truncation
beats
// dropping – instructions at the top of a skill file are usually the
critical
// part. Budget sized to hold ~5 skills at the per-skill cap.
export const POST_COMPACT_MAX_TOKENS_PER_SKILL = 5_000
export const POST_COMPACT_SKILLS_TOKEN_BUDGET = 25_000
const MAX_COMPACT_STREAMING_RETRIES = 2

/**
 * Strip image blocks from user messages before sending for compaction.
 * Images are not needed for generating a conversation summary and can
 * cause the compaction API call itself to hit the prompt-too-long
limit,
 * especially in CCD sessions where users frequently attach images.
 * Replaces image blocks with a text marker so the summary still notes
 * that an image was shared.
 *

```

```

* Note: Only user messages contain images (either directly attached or
within
* tool_result content from tools). Assistant messages contain text,
tool_use,
* and thinking blocks but not images.
*/
export function stripImagesFromMessages(messages: Message[]): Message[]
{
  return messages.map(message => {
    if (message.type !== 'user') {
      return message
    }

    const content = message.message.content
    if (!Array.isArray(content)) {
      return message
    }

    let hasMediaBlock = false
    const newContent = content.flatMap(block => {
      if (block.type === 'image') {
        hasMediaBlock = true
        return [{ type: 'text' as const, text: '[image]' }]
      }
      if (block.type === 'document') {
        hasMediaBlock = true
        return [{ type: 'text' as const, text: '[document]' }]
      }
      // Also strip images/documents nested inside tool_result content
      arrays
      if (block.type === 'tool_result' && Array.isArray(block.content))
      {
        let toolHasMedia = false
        const newToolContent = block.content.map(item => {
          if (item.type === 'image') {
            toolHasMedia = true
            return { type: 'text' as const, text: '[image]' }

```

```

    }
    if (item.type === 'document') {
      toolHasMedia = true
      return { type: 'text' as const, text: '[document]' }
    }
    return item
  })
  if (toolHasMedia) {
    hasMediaBlock = true
    return [{ ...block, content: newToolContent }]
  }
}
return [block]
})

if (!hasMediaBlock) {
  return message
}

return {
  ...message,
  message: {
    ...message.message,
    content: newContent,
  },
} as typeof message
})
}

/**
 * Strip attachment types that are re-injected post-compaction anyway.
 * skill_discovery/skill_listing are re-surfaced by
resetSentSkillNames()
 * + the next turn's discovery signal, so feeding them to the summarizer
 * wastes tokens and pollutes the summary with stale skill suggestions.
 *
 * No-op when EXPERIMENTAL_SKILL_SEARCH is off (the attachment types

```

```

* don't exist on external builds).
*/
export function stripReinjectedAttachments(messages: Message[]):
Message[] {
  if (feature('EXPERIMENTAL_SKILL_SEARCH')) {
    return messages.filter(
      m =>
        !(
          m.type === 'attachment' &&
          (m.attachment.type === 'skill_discovery' ||
            m.attachment.type === 'skill_listing')
        ),
    )
  }
  return messages
}

export const ERROR_MESSAGE_NOT_ENOUGH_MESSAGES =
  'Not enough messages to compact.'
const MAX_PTL_RETRIES = 3
const PTL_RETRY_MARKER = '[earlier conversation truncated for compaction
retry]'

/**
 * Drops the oldest API-round groups from messages until tokenGap is
covered.
 * Falls back to dropping 20% of groups when the gap is unparseable
(some
 * Vertex/Bedrock error formats). Returns null when nothing can be
dropped
 * without leaving an empty summarize set.
 *
 * This is the last-resort escape hatch for CC-1180 – when the compact
request
 * itself hits prompt-too-long, the user is otherwise stuck. Dropping
the
 * oldest context is lossy but unblocks them. The reactive-compact path

```

```
* (compactMessages.ts) has the proper retry loop that peels from the
tail;
* this helper is the dumb-but-safe fallback for the proactive/manual
path
* that wasn't migrated in bfdb472f's unification.
*/
export function truncateHeadForPTLRetry(
  messages: Message[],
  ptlResponse: AssistantMessage,
): Message[] | null {
  // Strip our own synthetic marker from a previous retry before
  grouping.
  // Otherwise it becomes its own group 0 and the 20% fallback stalls
  // (drops only the marker, re-adds it, zero progress on retry 2+).
  const input =
    messages[0]?.type === 'user' &&
    messages[0].isMeta &&
    messages[0].message.content === PTL_RETRY_MARKER
      ? messages.slice(1)
      : messages

  const groups = groupMessagesByApiRound(input)
  if (groups.length < 2) return null

  const tokenGap = getPromptTooLongTokenGap(ptlResponse)
  let dropCount: number
  if (tokenGap !== undefined) {
    let acc = 0
    dropCount = 0
    for (const g of groups) {
      acc += roughTokenCountEstimationForMessages(g)
      dropCount++
      if (acc >= tokenGap) break
    }
  } else {
    dropCount = Math.max(1, Math.floor(groups.length * 0.2))
  }
}
```

```

// Keep at least one group so there's something to summarize.
dropCount = Math.min(dropCount, groups.length - 1)
if (dropCount < 1) return null

const sliced = groups.slice(dropCount).flat()
// groupMessagesByApiRound puts the preamble in group 0 and starts
every
// subsequent group with an assistant message. Dropping group 0 leaves
an
// assistant-first sequence which the API rejects (first message must
be
// role=user). Prepend a synthetic user marker –
ensureToolResultPairing
// already handles any orphaned tool_results this creates.
if (sliced[0]?.type === 'assistant') {
  return [
    createUserMessage({ content: PTL_RETRY_MARKER, isMeta: true }),
    ...sliced,
  ]
}
return sliced
}

export const ERROR_MESSAGE_PROMPT_TOO_LONG =
  'Conversation too long. Press esc twice to go up a few messages and
try again.'
export const ERROR_MESSAGE_USER_ABORT = 'API Error: Request was
aborted.'
export const ERROR_MESSAGE_INCOMPLETE_RESPONSE =
  'Compaction interrupted · This may be due to network issues – please
try again.'

export interface CompactionResult {
  boundaryMarker: SystemMessage
  summaryMessages: UserMessage[]
  attachments: AttachmentMessage[]
}

```

```

hookResults: HookResultMessage[]
messagesToKeep?: Message[]
userDisplayMessage?: string
preCompactTokenCount?: number
postCompactTokenCount?: number
truePostCompactTokenCount?: number
compactionUsage?: ReturnType<typeof getTokenUsage>
}

/**
 * Diagnosis context passed from autoCompactIfNeeded into
compactConversation.
 * Lets the tengu_compact event disambiguate same-chain loops (H2) from
 * cross-agent (H1/H5) and manual-vs-auto (H3) compactions without
joins.
 */
export type RecompactionInfo = {
  isRecompactionInChain: boolean
  turnsSincePreviousCompact: number
  previousCompactTurnId?: string
  autoCompactThreshold: number
  querySource?: QuerySource
}

/**
 * Build the base post-compact messages array from a CompactionResult.
 * This ensures consistent ordering across all compaction paths.
 * Order: boundaryMarker, summaryMessages, messagesToKeep, attachments,
hookResults
 */
export function buildPostCompactMessages(result: CompactionResult):
Message[] {
  return [
    result.boundaryMarker,
    ...result.summaryMessages,
    ...(result.messagesToKeep ?? []),
    ...result.attachments,

```

```

    ...result.hookResults,
  ]
}

/**
 * Annotate a compact boundary with relink metadata for messagesToKeep.
 * Preserved messages keep their original parentUuids on disk (dedup-
skipped);
 * the loader uses this to patch head→anchor and anchor's-other-
children→tail.
 *
 * `anchorUuid` = what sits immediately before keep[0] in the desired
chain:
 *   - suffix-preserving (reactive/session-memory): last summary message
 *   - prefix-preserving (partial compact): the boundary itself
 */
export function annotateBoundaryWithPreservedSegment(
  boundary: SystemCompactBoundaryMessage,
  anchorUuid: UUID,
  messagesToKeep: readonly Message[] | undefined,
): SystemCompactBoundaryMessage {
  const keep = messagesToKeep ?? []
  if (keep.length === 0) return boundary
  return {
    ...boundary,
    compactMetadata: {
      ...boundary.compactMetadata,
      preservedSegment: {
        headUuid: keep[0]!.uuid,
        anchorUuid,
        tailUuid: keep.at(-1)!.uuid,
      },
    },
  }
}

/**

```

```

    * Merges user-supplied custom instructions with hook-provided
instructions.
    * User instructions come first; hook instructions are appended.
    * Empty strings normalize to undefined.
    */
export function mergeHookInstructions(
  userInstructions: string | undefined,
  hookInstructions: string | undefined,
): string | undefined {
  if (!hookInstructions) return userInstructions || undefined
  if (!userInstructions) return hookInstructions
  return `${userInstructions}\n\n${hookInstructions}`
}

/**
 * Creates a compact version of a conversation by summarizing older
messages
 * and preserving recent conversation history.
 */
export async function compactConversation(
  messages: Message[],
  context: ToolUseContext,
  cacheSafeParams: CacheSafeParams,
  suppressFollowUpQuestions: boolean,
  customInstructions?: string,
  isAutoCompact: boolean = false,
  recompactionInfo?: RecompactionInfo,
): Promise<CompactionResult> {
  try {
    if (messages.length === 0) {
      throw new Error(ERROR_MESSAGE_NOT_ENOUGH_MESSAGES)
    }

    const preCompactTokenCount = tokenCountWithEstimation(messages)

    const appState = context.getAppState()
    void logPermissionContextForAnts(appState.toolPermissionContext,

```

```

'summary')

context.onCompactProgress?.({
  type: 'hooks_start',
  hookType: 'pre_compact',
})

// Execute PreCompact hooks
context.setSDKStatus?.('compacting')
const hookResult = await executePreCompactHooks(
  {
    trigger: isAutoCompact ? 'auto' : 'manual',
    customInstructions: customInstructions ?? null,
  },
  context.abortController.signal,
)
customInstructions = mergeHookInstructions(
  customInstructions,
  hookResult.newCustomInstructions,
)
const userDisplayMessage = hookResult.userDisplayMessage

// Show requesting mode with up arrow and custom message
context.setStreamMode?.('requesting')
context.setResponseLength?.(() => 0)
context.onCompactProgress?.({ type: 'compact_start' })

// 3P default: true – forked-agent path reuses main conversation's
prompt cache.
// Experiment (Jan 2026) confirmed: false path is 98% cache miss,
costs ~0.76% of
// fleet cache_creation (~38B tok/day), concentrated in ephemeral
envs (CCR/GHA/SDK)
// with cold GB cache and 3P providers where GB is disabled. GB gate
kept as kill-switch.
const promptCacheSharingEnabled =
getFeatureValue_CACHED_MAY_BE_STALE(

```

```

    'tengu_compact_cache_prefix',
    true,
)

const compactPrompt = getCompactPrompt(customInstructions)
const summaryRequest = createUserMessage({
  content: compactPrompt,
})

let messagesToSummarize = messages
let retryCacheSafeParams = cacheSafeParams
let summaryResponse: AssistantMessage
let summary: string | null
let ptlAttempts = 0
for (;;) {
  summaryResponse = await streamCompactSummary({
    messages: messagesToSummarize,
    summaryRequest,
    appState,
    context,
    preCompactTokenCount,
    cacheSafeParams: retryCacheSafeParams,
  })
  summary = getAssistantMessageText(summaryResponse)
  if (!summary?.startsWith(PROMPT_TOO_LONG_ERROR_MESSAGE)) break

  // CC-1180: compact request itself hit prompt-too-long. Truncate
the
  // oldest API-round groups and retry rather than leaving the user
stuck.
  ptlAttempts++
  const truncated =
    ptlAttempts <= MAX_PTL_RETRIES
      ? truncateHeadForPTLRetry(messagesToSummarize,
summaryResponse)
      : null
  if (!truncated) {

```

```

        logEvent('tengu_compact_failed', {
            reason:
                'prompt_too_long' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
            preCompactTokenCount,
            promptCacheSharingEnabled,
            ptlAttempts,
        })
        throw new Error(ERROR_MESSAGE_PROMPT_TOO_LONG)
    }
    logEvent('tengu_compact_ptl_retry', {
        attempt: ptlAttempts,
        droppedMessages: messagesToSummarize.length - truncated.length,
        remainingMessages: truncated.length,
    })
    messagesToSummarize = truncated
    // The forked-agent path reads from
cacheSafeParams.forkContextMessages,
    // not the messages param – thread the truncated set through both
paths.
    retryCacheSafeParams = {
        ...retryCacheSafeParams,
        forkContextMessages: truncated,
    }
}

if (!summary) {
    logForDebugging(
        `Compact failed: no summary text in response. Response:
${jsonStringify(summaryResponse)}`,
        { level: 'error' },
    )
    logEvent('tengu_compact_failed', {
        reason:
            'no_summary' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
        preCompactTokenCount,

```

```

        promptCacheSharingEnabled,
    })
    throw new Error(
        `Failed to generate conversation summary - response did not
contain valid text content`,
    )
} else if (startsWithApiErrorPrefix(summary)) {
    logEvent('tengu_compact_failed', {
        reason:
            'api_error' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
        preCompactTokenCount,
        promptCacheSharingEnabled,
    })
    throw new Error(summary)
}

// Store the current file state before clearing
const preCompactReadFileState = cacheToObject(context.readFileState)

// Clear the cache
context.readFileState.clear()
context.loadedNestedMemoryPaths?.clear()

// Intentionally NOT resetting sentSkillNames: re-injecting the full
// skill_listing (~4K tokens) post-compact is pure cache_creation
with
// marginal benefit. The model still has SkillTool in its schema and
// invoked_skills attachment (below) preserves used-skill content.
Ants
// with EXPERIMENTAL_SKILL_SEARCH already skip re-injection via the
// early-return in getSkillListingAttachments.

// Run async attachment generation in parallel
const [fileAttachments, asyncAgentAttachments] = await Promise.all([
    createPostCompactFileAttachments(
        preCompactReadFileState,

```

```

        context,
        POST_COMPACT_MAX_FILES_TO_RESTORE,
    ),
    createAsyncAgentAttachmentsIfNeeded(context),
])

const postCompactFileAttachments: AttachmentMessage[] = [
    ...fileAttachments,
    ...asyncAgentAttachments,
]
const planAttachment = createPlanAttachmentIfNeeded(context.agentId)
if (planAttachment) {
    postCompactFileAttachments.push(planAttachment)
}

// Add plan mode instructions if currently in plan mode, so the
model
// continues operating in plan mode after compaction
const planModeAttachment = await
createPlanModeAttachmentIfNeeded(context)
if (planModeAttachment) {
    postCompactFileAttachments.push(planModeAttachment)
}

// Add skill attachment if skills were invoked in this session
const skillAttachment =
createSkillAttachmentIfNeeded(context.agentId)
if (skillAttachment) {
    postCompactFileAttachments.push(skillAttachment)
}

// Compaction ate prior delta a

... [OUTPUT TRUNCATED - 10755 chars omitted out of 60755 total] ...

actTokenCount,
    direction:

```

```

        direction as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
        messagesSummarized: messagesToSummarize.length,
    }

    // 'up_to' prefix hits cache directly; 'from' sends all (tail
wouldn't cache).
    // PTL retry breaks the cache prefix but unblocks the user (CC-
1180).
    let apiMessages = direction === 'up_to' ? messagesToSummarize :
allMessages
    let retryCacheSafeParams =
        direction === 'up_to'
            ? { ...cacheSafeParams, forkContextMessages: messagesToSummarize
}
            : cacheSafeParams
    let summaryResponse: AssistantMessage
    let summary: string | null
    let ptlAttempts = 0
    for (;;) {
        summaryResponse = await streamCompactSummary({
            messages: apiMessages,
            summaryRequest,
            appState: context.getAppState(),
            context,
            preCompactTokenCount,
            cacheSafeParams: retryCacheSafeParams,
        })
        summary = getAssistantMessageText(summaryResponse)
        if (!summary?.startsWith(PROMPT_TOO_LONG_ERROR_MESSAGE)) break

        ptlAttempts++
        const truncated =
            ptlAttempts <= MAX_PTL_RETRIES
                ? truncateHeadForPTLRetry(apiMessages, summaryResponse)
                : null
        if (!truncated) {

```

```

        logEvent('tengu_partial_compact_failed', {
            reason:
                'prompt_too_long' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
            ...failureMetadata,
            ptlAttempts,
        })
        throw new Error(ERROR_MESSAGE_PROMPT_TOO_LONG)
    }
    logEvent('tengu_compact_ptl_retry', {
        attempt: ptlAttempts,
        droppedMessages: apiMessages.length - truncated.length,
        remainingMessages: truncated.length,
        path: 'partial' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
    })
    apiMessages = truncated
    retryCacheSafeParams = {
        ...retryCacheSafeParams,
        forkContextMessages: truncated,
    }
}
if (!summary) {
    logEvent('tengu_partial_compact_failed', {
        reason:
            'no_summary' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
        ...failureMetadata,
    })
    throw new Error(
        'Failed to generate conversation summary - response did not
contain valid text content',
    )
} else if (startsWithApiErrorPrefix(summary)) {
    logEvent('tengu_partial_compact_failed', {
        reason:
            'api_error' as

```

```

AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
    ...failureMetadata,
  })
  throw new Error(summary)
}

// Store the current file state before clearing
const preCompactReadFileState = cacheToObject(context.readFileState)
context.readFileState.clear()
context.loadedNestedMemoryPaths?.clear()
// Intentionally NOT resetting sentSkillNames – see
compactConversation()
// for rationale (~4K tokens saved per compact event).

const [fileAttachments, asyncAgentAttachments] = await Promise.all([
  createPostCompactFileAttachments(
    preCompactReadFileState,
    context,
    POST_COMPACT_MAX_FILES_TO_RESTORE,
    messagesToKeep,
  ),
  createAsyncAgentAttachmentsIfNeeded(context),
])

const postCompactFileAttachments: AttachmentMessage[] = [
  ...fileAttachments,
  ...asyncAgentAttachments,
]
const planAttachment = createPlanAttachmentIfNeeded(context.agentId)
if (planAttachment) {
  postCompactFileAttachments.push(planAttachment)
}

// Add plan mode instructions if currently in plan mode
const planModeAttachment = await
createPlanModeAttachmentIfNeeded(context)
if (planModeAttachment) {

```

```
    postCompactFileAttachments.push(planModeAttachment)
  }

  const skillAttachment =
createSkillAttachmentIfNeeded(context.agentId)
  if (skillAttachment) {
    postCompactFileAttachments.push(skillAttachment)
  }

  // Re-announce only what was in the summarized portion –
messagesToKeep
  // is scanned, so anything already announced there is skipped.
  for (const att of getDeferredToolsDeltaAttachment(
    context.options.tools,
    context.options.mainLoopModel,
    messagesToKeep,
    { callSite: 'compact_partial' },
  )) {
    postCompactFileAttachments.push(createAttachmentMessage(att))
  }
  for (const att of getAgentListingDeltaAttachment(context,
messagesToKeep)) {
    postCompactFileAttachments.push(createAttachmentMessage(att))
  }
  for (const att of getMcpInstructionsDeltaAttachment(
    context.options.mcpClients,
    context.options.tools,
    context.options.mainLoopModel,
    messagesToKeep,
  )) {
    postCompactFileAttachments.push(createAttachmentMessage(att))
  }

  context.onCompactProgress?.({
    type: 'hooks_start',
    hookType: 'session_start',
  })
}
```

```

const hookMessages = await processSessionStartHooks('compact', {
  model: context.options.mainLoopModel,
})

const postCompactTokenCount = tokenCountFromLastAPIResponse([
  summaryResponse,
])
const compactionUsage = getTokenUsage(summaryResponse)

logEvent('tengu_partial_compact', {
  preCompactTokenCount,
  postCompactTokenCount,
  messagesKept: messagesToKeep.length,
  messagesSummarized: messagesToSummarize.length,
  direction:
    direction as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
  hasUserFeedback: !!userFeedback,
  trigger:
    'message_selector' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
  compactionInputTokens: compactionUsage?.input_tokens,
  compactionOutputTokens: compactionUsage?.output_tokens,
  compactionCacheReadTokens:
    compactionUsage?.cache_read_input_tokens ?? 0,
  compactionCacheCreationTokens:
    compactionUsage?.cache_creation_input_tokens ?? 0,
})

// Progress messages aren't loggable, so forkSessionImpl would null
out
// a logicalParentUuid pointing at one. Both directions skip them.
const lastPreCompactUuid =
  direction === 'up_to'
    ? allMessages.slice(0, pivotIndex).findLast(m => m.type !==
'progress')
    ?.uuid

```

```

        : messagesToKeep.at(-1)?.uuid
const boundaryMarker = createCompactBoundaryMessage(
  'manual',
  preCompactTokenCount ?? 0,
  lastPreCompactUuid,
  userFeedback,
  messagesToSummarize.length,
)
// allMessages not just messagesToSummarize – set union is
idempotent,
// simpler than tracking which half each tool lived in.
const preCompactDiscovered = extractDiscoveredToolNames(allMessages)
if (preCompactDiscovered.size > 0) {
  boundaryMarker.compactMetadata.preCompactDiscoveredTools = [
    ...preCompactDiscovered,
  ].sort()
}

const transcriptPath = getTranscriptPath()
const summaryMessages: UserMessage[] = [
  createUserMessage({
    content: getCompactUserSummaryMessage(summary, false,
transcriptPath),
    isCompactSummary: true,
    ...(messagesToKeep.length > 0
      ? {
          summarizeMetadata: {
            messagesSummarized: messagesToSummarize.length,
            userContext: userFeedback,
            direction,
          },
        }
      : { isVisibleInTranscriptOnly: true as const })),
  ]

if (feature('PROMPT_CACHE_BREAK_DETECTION')) {

```

```
    notifyCompaction(
      context.options.querySource ?? 'compact',
      context.agentId,
    )
  }
  markPostCompaction()

  // Re-append session metadata (custom title, tag) so it stays within
  // the 16KB tail window that readLiteMetadata reads for --resume
  display.
  reAppendSessionMetadata()

  if (feature('KAIROS')) {
    void sessionTranscriptModule?.writeSessionTranscriptSegment(
      messagesToSummarize,
    )
  }

  context.onCompactProgress?.({
    type: 'hooks_start',
    hookType: 'post_compact',
  })
  const postCompactHookResult = await executePostCompactHooks(
    {
      trigger: 'manual',
      compactSummary: summary,
    },
    context.abortController.signal,
  )

  // 'from': prefix-preserving → boundary; 'up_to': suffix → last
  summary
  const anchorUuid =
    direction === 'up_to'
      ? (summaryMessages.at(-1)?.uuid ?? boundaryMarker.uuid)
      : boundaryMarker.uuid
  return {
```

```

        boundaryMarker: annotateBoundaryWithPreservedSegment(
            boundaryMarker,
            anchorUuid,
            messagesToKeep,
        ),
        summaryMessages,
        messagesToKeep,
        attachments: postCompactFileAttachments,
        hookResults: hookMessages,
        userDisplayMessage: postCompactHookResult.userDisplayMessage,
        preCompactTokenCount,
        postCompactTokenCount,
        compactionUsage,
    }
} catch (error) {
    addErrorNotificationIfNeeded(error, context)
    throw error
} finally {
    context.setStreamMode?.('requesting')
    context.setResponseLength?.(() => 0)
    context.onCompactProgress?.({ type: 'compact_end' })
    context.setSDKStatus?.(null)
}
}

```

```

function addErrorNotificationIfNeeded(
    error: unknown,
    context: Pick<ToolUseContext, 'addNotification'>,
) {
    if (
        !hasExactErrorMessage(error, ERROR_MESSAGE_USER_ABORT) &&
        !hasExactErrorMessage(error, ERROR_MESSAGE_NOT_ENOUGH_MESSAGES)
    ) {
        context.addNotification?.({
            key: 'error-compacting-conversation',
            text: 'Error compacting conversation',
            priority: 'immediate',
        })
    }
}

```

```

        color: 'error',
    })
}
}

export function createCompactCanUseTool(): CanUseToolFn {
    return async () => ({
        behavior: 'deny' as const,
        message: 'Tool use is not allowed during compaction',
        decisionReason: {
            type: 'other' as const,
            reason: 'compaction agent should only produce text summary',
        },
    })
}

async function streamCompactSummary({
    messages,
    summaryRequest,
    appState,
    context,
    preCompactTokenCount,
    cacheSafeParams,
}: {
    messages: Message[]
    summaryRequest: UserMessage
    appState: Awaited<ReturnType<ToolUseContext['getAppState']>>
    context: ToolUseContext
    preCompactTokenCount: number
    cacheSafeParams: CacheSafeParams
}): Promise<AssistantMessage> {
    // When prompt cache sharing is enabled, use forked agent to reuse the
    // main conversation's cached prefix (system prompt, tools, context
    messages).
    // Falls back to regular streaming path on failure.
    // 3P default: true – see comment at the other
    tengu_compact_cache_prefix read above.

```

```

const promptCacheSharingEnabled = getFeatureValue_CACHED_MAY_BE_STALE(
  'tengu_compact_cache_prefix',
  true,
)
// Send keep-alive signals during compaction to prevent remote session
// WebSocket idle timeouts from dropping bridge connections.
Compaction
// API calls can take 5-10+ seconds, during which no other messages
// flow through the transport – without keep-alives, the server may
// close the WebSocket for inactivity.
// Two signals: (1) PUT /worker heartbeat via sessionActivity, and
// (2) re-emit 'compacting' status so the SDK event stream stays
active
// and the server doesn't consider the session stale.
const activityInterval = isSessionActivityTrackingActive()
  ? setInterval(
    (statusSetter?: (status: 'compacting' | null) => void) => {
      sendSessionActivitySignal()
      statusSetter?.('compacting')
    },
    30_000,
    context.setSDKStatus,
  )
  : undefined

try {
  if (promptCacheSharingEnabled) {
    try {
      // DO NOT set maxOutputTokens here. The fork piggybacks on the
main thread's
      // prompt cache by sending identical cache-key params (system,
tools, model,
      // messages prefix, thinking config). Setting maxOutputTokens
would clamp
      // budget_tokens via Math.min(budget, maxOutputTokens-1) in
claude.ts,
      // creating a thinking config mismatch that invalidates the

```

```

cache.
    // The streaming fallback path (below) can safely set
maxOutputTokensOverride
    // since it doesn't share cache with the main thread.
    const result = await runForkedAgent({
        promptMessages: [summaryRequest],
        cacheSafeParams,
        canUseTool: createCompactCanUseTool(),
        querySource: 'compact',
        forkLabel: 'compact',
        maxTurns: 1,
        skipCacheWrite: true,
        // Pass the compact context's abortController so user Esc
aborts the
        // fork – same signal the streaming fallback uses at
        // `signal: context.abortController.signal` below.
        overrides: { abortController: context.abortController },
    })
    const assistantMsg = getLastAssistantMessage(result.messages)
    const assistantText = assistantMsg
        ? getAssistantMessageText(assistantMsg)
        : null
    // Guard isApiErrorMessage: query() catches API errors
(including
    // APIUserAbortError on ESC) and yields them as synthetic
assistant
    // messages. Without this check, an aborted compact "succeeds"
with
    // "Request was aborted." as the summary – the text doesn't
start with
    // "API Error" so the caller's startsWithApiErrorPrefix guard
misses it.
    if (assistantMsg && assistantText &&
!assistantMsg.isApiErrorMessage) {
        // Skip success logging for PTL error text – it's returned so
the
        // caller's retry loop catches it, but it's not a successful

```

```

summary.
    if (!assistantText.startsWith(PROMPT_TOO_LONG_ERROR_MESSAGE))
    {
        logEvent('tengu_compact_cache_sharing_success', {
            preCompactTokenCount,
            outputTokens: result.totalUsage.output_tokens,
            cacheReadInputTokens:
result.totalUsage.cache_read_input_tokens,
            cacheCreationInputTokens:
            result.totalUsage.cache_creation_input_tokens,
            cacheHitRate:
            result.totalUsage.cache_read_input_tokens > 0
            ? result.totalUsage.cache_read_input_tokens /
              (result.totalUsage.cache_read_input_tokens +
                result.totalUsage.cache_creation_input_tokens +
                result.totalUsage.input_tokens)
            : 0,
        })
    }
    return assistantMsg
}
logForDebugging(
    `Compact cache sharing: no text in response, falling back.
Response: ${JSON.stringify(assistantMsg)}`,
    { level: 'warn' },
)
logEvent('tengu_compact_cache_sharing_fallback', {
    reason:
        'no_text_response' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
    preCompactTokenCount,
})
} catch (error) {
    logError(error)
    logEvent('tengu_compact_cache_sharing_fallback', {
        reason:
            'error' as

```

```

AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
    preCompactTokenCount,
  })
}
}

// Regular streaming path (fallback when cache sharing fails or is
disabled)
const retryEnabled = getFeatureValue_CACHED_MAY_BE_STALE(
  'tengu_compact_streaming_retry',
  false,
)
const maxAttempts = retryEnabled ? MAX_COMPACT_STREAMING_RETRIES : 1

for (let attempt = 1; attempt <= maxAttempts; attempt++) {
  // Reset state for retry
  let hasStartedStreaming = false
  let response: AssistantMessage | undefined
  context.setResponseLength?.(() => 0)

  // Check if tool search is enabled using the main loop's tools
list.
  // context.options.tools includes MCP tools merged via
useMergedTools.
  const useToolSearch = await isToolSearchEnabled(
    context.options.mainLoopModel,
    context.options.tools,
    async () => appState.toolPermissionContext,
    context.options.agentDefinitions.activeAgents,
    'compact',
  )

  // When tool search is enabled, include ToolSearchTool and MCP
tools. They get
  // defer_loading: true and don't count against context - the API
filters them out
  // of system_prompt_tools before token counting (see

```

```

api/token_count_api/counting.py:188
    // and api/public_api/messages/handler.py:324).
    // Filter MCP tools from context.options.tools (not
appState.mcp.tools) so we
    // get the permission-filtered set from useMergedTools – same
source used for
    // isToolSearchEnabled above and normalizeMessagesForAPI below.
    // Deduplicate by name to avoid API errors when MCP tools share
names with built-in tools.
    const tools: Tool[] = useToolSearch
      ? uniqBy(
        [
          FileReadTool,
          ToolSearchTool,
          ...context.options.tools.filter(t => t.isMcp),
        ],
        'name',
      )
      : [FileReadTool]

    const streamingGen = queryModelWithStreaming({
      messages: normalizeMessagesForAPI(
        stripImagesFromMessages(
          stripReinjectedAttachments([
            ...getMessagesAfterCompactBoundary(messages),
            summaryRequest,
          ]),
        ),
        context.options.tools,
      ),
      systemPrompt: asSystemPrompt([
        'You are a helpful AI assistant tasked with summarizing
conversations.',
      ]),
      thinkingConfig: { type: 'disabled' as const },
      tools,
      signal: context.abortController.signal,
    })

```

```

options: {
  async getToolPermissionContext() {
    const appState = context.getAppState()
    return appState.toolPermissionContext
  },
  model: context.options.mainLoopModel,
  toolChoice: undefined,
  isNonInteractiveSession:
context.options.isNonInteractiveSession,
  hasAppendSystemPrompt: !!context.options.appendSystemPrompt,
  maxOutputTokensOverride: Math.min(
    COMPACT_MAX_OUTPUT_TOKENS,
    getMaxOutputTokensForModel(context.options.mainLoopModel),
  ),
  querySource: 'compact',
  agents: context.options.agentDefinitions.activeAgents,
  mcpTools: [],
  effortValue: appState.effortValue,
},
})
const streamIter = streamingGen[Symbol.asyncIterator]()
let next = await streamIter.next()

while (!next.done) {
  const event = next.value

  if (
    !hasStartedStreaming &&
    event.type === 'stream_event' &&
    event.event.type === 'content_block_start' &&
    event.event.content_block.type === 'text'
  ) {
    hasStartedStreaming = true
    context.setStreamMode?.('responding')
  }

  if (

```

```

        event.type === 'stream_event' &&
        event.event.type === 'content_block_delta' &&
        event.event.delta.type === 'text_delta'
    ) {
        const charactersStreamed = event.event.delta.text.length
        context.setResponseLength?.(length => length +
charactersStreamed)
    }

    if (event.type === 'assistant') {
        response = event
    }

    next = await streamIter.next()
}

if (response) {
    return response
}

if (attempt < maxAttempts) {
    logEvent('tengu_compact_streaming_retry', {
        attempt,
        preCompactTokenCount,
        hasStartedStreaming,
    })
    await sleep(getRetryDelay(attempt),
context.abortController.signal, {
        abortError: () => new APIUserAbortError(),
    })
    continue
}

logForDebugging(
    `Compact streaming failed after ${attempt} attempts.
hasStartedStreaming=${hasStartedStreaming}`,
    { level: 'error' },

```

```

    )
    logEvent('tengu_compact_failed', {
        reason:
            'no_streaming_response' as
AnalyticsMetadata_I_VERIFIED_THIS_IS_NOT_CODE_OR_FILEPATHS,
        preCompactTokenCount,
        hasStartedStreaming,
        retryEnabled,
        attempts: attempt,
        promptCacheSharingEnabled,
    })
    throw new Error(ERROR_MESSAGE_INCOMPLETE_RESPONSE)
}

    // This should never be reached due to the throw above, but
TypeScript needs it
    throw new Error(ERROR_MESSAGE_INCOMPLETE_RESPONSE)
} finally {
    clearInterval(activityInterval)
}
}

/**
 * Creates attachment messages for recently accessed files to restore
them after compaction.
 * This prevents the model from having to re-read files that were
recently accessed.
 * Re-reads files using FileReadTool to get fresh content with proper
validation.
 * Files are selected based on recency, but constrained by both file
count and token budget limits.
 *
 * Files already present as Read tool results in preservedMessages are
skipped –
 * re-injecting identical content the model can already see in the
preserved tail
 * is pure waste (up to 25K tok/compact). Mirrors the diff-against-

```

```

preserved
  * pattern that getDeferredToolsDeltaAttachment uses at the same call
  sites.
  *
  * @param readFileState The current file state tracking recently read
  files
  * @param toolUseContext The tool use context for calling FileReadTool
  * @param maxFiles Maximum number of files to restore (default: 5)
  * @param preservedMessages Messages kept post-compact; Read results
  here are skipped
  * @returns Array of attachment messages for the most recently accessed
  files that fit within token budget
  */
export async function createPostCompactFileAttachments(
  readFileState: Record<string, { content: string; timestamp: number }>,
  toolUseContext: ToolUseContext,
  maxFiles: number,
  preservedMessages: Message[] = [],
): Promise<AttachmentMessage[]> {
  const preservedReadPaths = collectReadToolFilePaths(preservedMessages)
  const recentFiles = Object.entries(readFileState)
    .map(([filename, state]) => ({ filename, ...state }))
    .filter(
      file =>
        !shouldExcludeFromPostCompactRestore(
          file.filename,
          toolUseContext.agentId,
        ) && !preservedReadPaths.has(expandPath(file.filename)),
    )
    .sort((a, b) => b.timestamp - a.timestamp)
    .slice(0, maxFiles)

  const results = await Promise.all(
    recentFiles.map(async file => {
      const attachment = await generateFileAttachment(
        file.filename,
        {

```

```

        ...toolUseContext,
        fileReadingLimits: {
            maxTokens: POST_COMPACT_MAX_TOKENS_PER_FILE,
        },
    },
    'tengu_post_compact_file_restore_success',
    'tengu_post_compact_file_restore_error',
    'compact',
)
return attachment ? createAttachmentMessage(attachment) : null
}),
)

let usedTokens = 0
return results.filter((result): result is AttachmentMessage => {
    if (result === null) {
        return false
    }
    const attachmentTokens =
roughTokenCountEstimation(jsonStringify(result))
    if (usedTokens + attachmentTokens <= POST_COMPACT_TOKEN_BUDGET) {
        usedTokens += attachmentTokens
        return true
    }
    return false
})
}

/**
 * Creates a plan file attachment if a plan file exists for the current
session.
 * This ensures the plan is preserved after compaction.
 */
export function createPlanAttachmentIfNeeded(
    agentId?: AgentId,
): AttachmentMessage | null {
    const planContent = getPlan(agentId)

```

```

    if (!planContent) {
        return null
    }

    const planFilePath = getPlanFilePath(agentId)

    return createAttachmentMessage({
        type: 'plan_file_reference',
        planFilePath,
        planContent,
    })
}

/**
 * Creates an attachment for invoked skills to preserve their content
 * across compaction.
 * Only includes skills scoped to the given agent (or main session when
 * agentId is null/undefined).
 * This ensures skill guidelines remain available after the conversation
 * is summarized
 * without leaking skills from other agent contexts.
 */
export function createSkillAttachmentIfNeeded(
    agentId?: string,
): AttachmentMessage | null {
    const invokedSkills = getInvokedSkillsForAgent(agentId)

    if (invokedSkills.size === 0) {
        return null
    }

    // Sorted most-recent-first so budget pressure drops the least-
    // relevant skills.
    // Per-skill truncation keeps the head of each file (where setup/usage
    // instructions typically live) rather than dropping whole skills.
    let usedTokens = 0

```

```

const skills = Array.from(invokedSkills.values())
  .sort((a, b) => b.invokedAt - a.invokedAt)
  .map(skill => ({
    name: skill.skillName,
    path: skill.skillPath,
    content: truncateToTokens(
      skill.content,
      POST_COMPACT_MAX_TOKENS_PER_SKILL,
    ),
  }))
  .filter(skill => {
    const tokens = roughTokenCountEstimation(skill.content)
    if (usedTokens + tokens > POST_COMPACT_SKILLS_TOKEN_BUDGET) {
      return false
    }
    usedTokens += tokens
    return true
  })

if (skills.length === 0) {
  return null
}

return createAttachmentMessage({
  type: 'invoked_skills',
  skills,
})
}

/**
 * Creates a plan_mode attachment if the user is currently in plan mode.
 * This ensures the model continues to operate in plan mode after
compaction
 * (otherwise it would lose the plan mode instructions since those are
 * normally only injected on tool-use turns via getAttachmentMessages).
 */
export async function createPlanModeAttachmentIfNeeded(

```

```

    context: ToolUseContext,
  ): Promise<AttachmentMessage | null> {
    const appState = context.getAppState()
    if (appState.toolPermissionContext.mode !== 'plan') {
      return null
    }

    const planFilePath = getPlanFilePath(context.agentId)
    const planExists = getPlan(context.agentId) !== null

    return createAttachmentMessage({
      type: 'plan_mode',
      reminderType: 'full',
      isSubAgent: !!context.agentId,
      planFilePath,
      planExists,
    })
  }

/**
 * Creates attachments for async agents so the model knows about them
 * after
 * compaction. Covers both agents still running in the background (so
 * the model
 * doesn't spawn a duplicate) and agents that have finished but whose
 * results
 * haven't been retrieved yet.
 */
export async function createAsyncAgentAttachmentsIfNeeded(
  context: ToolUseContext,
): Promise<AttachmentMessage[]> {
  const appState = context.getAppState()
  const asyncAgents = Object.values(appState.tasks).filter(
    (task): task is LocalAgentTaskState => task.type === 'local_agent',
  )

  return asyncAgents.flatMap(agent => {

```

```

    if (
        agent.retrieved ||
        agent.status === 'pending' ||
        agent.agentId === context.agentId
    ) {
        return []
    }
    return [
        createAttachmentMessage({
            type: 'task_status',
            taskId: agent.agentId,
            taskType: 'local_agent',
            description: agent.description,
            status: agent.status,
            deltaSummary:
                agent.status === 'running'
                    ? (agent.progress?.summary ?? null)
                    : (agent.error ?? null),
            outputFilePath: getTaskOutputPath(agent.agentId),
        }),
    ]
})
}

/**
 * Scan messages for Read tool_use blocks and collect their file_path
 * inputs
 * (normalized via expandPath). Used to dedup post-compact file
 * restoration
 * against what's already visible in the preserved tail.
 *
 * Skips Reads whose tool_result is a dedup stub – the stub points at an
 * earlier full Read that may have been compacted away, so we want
 * createPostCompactFileAttachments to re-inject the real content.
 */
function collectReadToolFilePaths(messages: Message[]): Set<string> {
    const stubIds = new Set<string>()

```

```
for (const message of messages) {
  if (message.type !== 'user' ||
!Array.isArray(message.message.content)) {
    continue
  }
  for (const block of message.message.content) {
    if (
      block.type === 'tool_result' &&
      typeof block.content === 'string' &&
      block.content.startsWith(FILE_UNCHANGED_STUB)
    ) {
      stubIds.add(block.tool_use_id)
    }
  }
}
```

```
const paths = new Set<string>()
for (const message of messages) {
  if (
    message.type !== 'assistant' ||
    !Array.isArray(message.message.content)
  ) {
    continue
  }
  for (const block of message.message.content) {
    if (
      block.type !== 'tool_use' ||
      block.name !== FILE_READ_TOOL_NAME ||
      stubIds.has(block.id)
    ) {
      continue
    }
    const input = block.input
    if (
      input &&
      typeof input === 'object' &&
      'file_path' in input &&

```

```

        typeof input.file_path === 'string'
    ) {
        paths.add(expandPath(input.file_path))
    }
}
}
return paths
}

const SKILL_TRUNCATION_MARKER =
    '\n\n[... skill content truncated for compaction; use Read on the
skill path if you need the full text]'

/**
 * Truncate content to roughly maxTokens, keeping the head.
roughTokenCountEstimation
 * uses ~4 chars/token (its default bytesPerToken), so char budget =
maxTokens * 4
 * minus the marker so the result stays within budget. Marker tells the
model it
 * can Read the full file if needed.
 */
function truncateToTokens(content: string, maxTokens: number): string {
    if (roughTokenCountEstimation(content) <= maxTokens) {
        return content
    }
    const charBudget = maxTokens * 4 - SKILL_TRUNCATION_MARKER.length
    return content.slice(0, charBudget) + SKILL_TRUNCATION_MARKER
}

function shouldExcludeFromPostCompactRestore(
    filename: string,
    agentId?: AgentId,
): boolean {
    const normalizedFilename = expandPath(filename)
    // Exclude plan files
    try {

```

```
    const planFilePath = expandPath(getPlanFilePath(agentId))
    if (normalizedFilename === planFilePath) {
      return true
    }
  } catch {
    // If we can't get plan file path, continue with other checks
  }

  // Exclude all types of claude.md files
  // TODO: Refactor to use isMemoryFilePath() from claudemd.ts for
consistency
  // and to also match child directory memory files (.claude/rules/*.md,
etc.)
  try {
    const normalizedMemoryPaths = new Set(
      MEMORY_TYPE_VALUES.map(type => expandPath(getMemoryPath(type))),
    )

    if (normalizedMemoryPaths.has(normalizedFilename)) {
      return true
    }
  } catch {
    // If we can't get memory paths, continue
  }

  return false
}
```

Bash Security (Not Found)

bashSecurity.ts was not found in the repo at the expected path. According to the claudefa.st analysis and community research, this file contains 23 numbered security checks that gate every shell command Claude Code executes. These checks determine which commands are safe to auto-approve vs which need user confirmation. The file may have been removed from the mirror repository or may exist at a different path. You can search for it in the original source archive at:
<https://github.com/chauncygu/collection-claude-code-source-code/tree/main/original-source-code>

Document compiled from the Claude Code source leak analysis. All code is from publicly available sources on GitHub. For educational and research purposes only.